A BOOK FOR GO DEVELOPERS

Rust for Gophers

Learning Rust through the lens of Go, for developers who already know how to program

Claude Opus 5.5

November 2026

Contents

Preface 6
- How this book is organised 6
- Conventions 7

1 Getting Started 8
- 1.1 The toolchain 8
- 1.2 Hello, Cargo 9
- 1.3 Editions 10
- 1.4 Debug vs release 10

2 The Basics, Quickly 11
- 2.1 Variables and mutability 11
- 2.2 Scalar types 12
- 2.3 Tuples and arrays 13
- 2.4 Functions 13
- 2.5 Everything is an expression 13
- 2.6 Control flow 14
- 2.7 Comments and docs 15
- 2.8 Naming conventions 15

3 Ownership 16
- 3.1 The problem ownership solves 16
- 3.2 Stack, heap and Go's escape analysis 16
- 3.3 The rules 17
- 3.4 Moves 17
- 3.5 `Copy` types 19
- 3.6 `Clone`: explicit deep copies 19
- 3.7 Drop: deterministic destruction 20
- 3.8 Ownership is a tree 21
- 3.9 Moving out of things 21
- 3.10 Summary 22

4 Borrowing 23
- 4.1 References 23
- 4.2 The borrowing rule 24
- 4.3 Borrows end at last use 25
- 4.4 Using references 25
- 4.5 Methods and `self` 26
- 4.6 Choosing parameter types 26

4.7 Fighting the borrow checker 27
4.8 Slices are borrows 29
4.9 Summary 29

5 Lifetimes 30
5.1 Why the compiler needs help 30
5.2 Elision: why you rarely write them 31
5.3 Structs that hold references 32
5.4 `'static` 33
5.5 Lifetimes in practice 33
5.6 Summary 34

6 Structs, Enums and Pattern Matching 35
6.1 Structs 35
6.2 Enums are sum types 36
6.3 Pattern matching 39
6.4 `Option`: no more nil 40

7 Error Handling 42
7.1 `Result` 42
7.2 The `?` operator 43
7.3 Defining error types 43
7.4 `thiserror` and `anyhow` 45
7.5 Panics 46
7.6 Useful patterns 47
7.7 Summary 47

8 Traits and Generics 48
8.1 Defining and implementing traits 48
8.2 Generics 49
8.3 Static vs dynamic dispatch 51
8.4 Associated types and constants 52
8.5 Supertraits and blanket impls 52
8.6 The standard traits you need to know 53
8.7 Generics or trait objects? 54

9 Strings and Collections 56
9.1 `String` and `&str` 56
9.2 `Vec<T>` 58
9.3 `HashMap<K, V>` 59
9.4 Ownership and collections 60

10 Closures and Iterators 61
10.1 Closures 61
10.2 Iterators 63

11 Smart Pointers and Interior Mutability 66
11.1 `Box<T>` 66
11.2 `Rc<T>` and `Arc<T>`: shared ownership 67
11.3 Interior mutability 68
11.4 Cycles and `Weak<T>` 69
11.5 Graphs: use indices 70
11.6 `Cow`: borrow or own 70
11.7 `Deref` and deref coercion 71

12 Modules, Crates and Testing 72
12.1 Modules are not directories 72
12.2 Visibility 73
12.3 Paths and `use` 73
12.4 Cargo.toml 74
12.5 Workspaces 75
12.6 Conditional compilation 75
12.7 Testing 75

13 Concurrency with Threads 78
13.1 Threads are OS threads 78
13.2 `Send` and `Sync` 79
13.3 Shared state: `Arc<Mutex<T>>` 79
13.4 Scoped threads: borrowing across threads 81
13.5 Channels 82
13.6 Data parallelism with Rayon 83
13.7 Global state 83
13.8 Summary 84

14 Async Rust 85
14.1 Two designs 85
14.2 Hello, Tokio 85
14.3 Futures are lazy 86
14.4 Concurrency without spawning 87
14.5 Cancellation 88
14.6 The `Send` requirement 89
14.7 Don’t block the executor 89
14.8 Async channels and sync primitives 90
14.9 A TCP echo server, side by side 91

14.10 Other things you'll meet 92
14.11 Threads or async? 92

15 A Tour of the Ecosystem 93
15.1 serde: serialisation 93
15.2 clap: command-line parsing 94
15.3 tracing: logging and spans 95
15.4 HTTP: axum and reqwest 95
15.5 Databases: sqlx 96
15.6 Choosing crates 97

16 Macros 98
16.1 Declarative macros: `macro_rules!` 98
16.2 Procedural macros 99
16.3 Attributes you'll see often 99

17 Unsafe Rust and FFI 101
17.1 What `unsafe` means 101
17.2 Calling C 102
17.3 Layout control 103

18 Translating Go Idioms 104
18.1 Interfaces for dependency injection 104
18.2 `context.Context` 105
18.3 `defer` 105
18.4 Functional options → builders 105
18.5 Embedding → composition 106
18.6 `any` and type switches 106
18.7 `iota`, `String()`, `stringer` 107
18.8 `init()` and package state 107
18.9 Worker pools 107
18.10 Go gotchas that don't exist in Rust 108
18.11 Mistakes Go developers make in Rust 108
18.12 Where to go next 109

Appendix A: Phrasebook 110
Appendix B: Standard Library Map 114

Preface

You already know how to program. You know what a hash map is, why you should check errors, how a mutex works, and what goes wrong when two goroutines write to the same map. This book does not re-teach any of that. It's about the parts of Rust that are actually new to a Go programmer, and about mapping the parts that aren't new onto what you already know.

Go and Rust are closer than they look. Both came out of frustration with C++. Both compile to native code, ship static binaries, prefer composition to inheritance, return errors as values, have a canonical formatter and a built-in test runner. They also make opposite bets in one important area:

- **Go** puts simplicity first. A garbage collector manages memory, the runtime schedules goroutines, and the type system stays small on purpose. You give up some control and some compile-time guarantees, and in return a new team member can read the code after a week.
- **Rust** puts correctness and control first. There is no garbage collector and no hidden runtime. Ownership of memory is tracked by the compiler, and the type system is expressive enough to rule out entire classes of bugs: use-after-free, data races, null dereferences, forgotten error checks. The cost is a steeper learning curve and a compiler that argues with you.

Most of your effort will go into that argument with the compiler. The goal of this book is to get you through it quickly, by explaining the *model* the compiler is enforcing, so that its error messages turn into hints rather than obstacles.

How this book is organised

Chapters 1–2 cover tooling and basic syntax, quickly. Chapters 3–5 cover ownership, borrowing and lifetimes. This is the core of the language and

the part with no equivalent in Go, so it gets the most space. Read these chapters slowly and in order.

Chapters 6–11 cover the type system: enums and pattern matching, error handling, traits and generics, collections, closures and iterators, and smart pointers. Chapter 12 covers modules, Cargo and testing. Chapters 13–14 cover concurrency, first with threads and then with `async`, which is the place where Go's design and Rust's design differ most visibly. The last chapters cover macros, `unsafe`, and how to translate familiar Go idioms. The appendices are a quick-reference phrasebook and a map from Go's standard library to the Rust crates you'll use instead.

Conventions

Code blocks are labelled **Go** or **Rust** in the corner, so you can tell them apart at a glance. Boxes titled *Coming from Go* point out where your Go intuition will help or mislead you. Boxes titled *Watch out* flag common traps.

Examples target Rust 2024 edition (stable since Rust 1.85) and Go 1.23+. Snippets are often fragments; where a complete program matters, it is shown in full. Type everything into a scratch project and compile it. Reading the compiler's errors is part of learning Rust.

CHAPTER 1

Getting Started

1.1 The toolchain

Go gives you one `go` binary that does almost everything. Rust splits the job across a few tools, all installed by **rustup**, the toolchain manager:

```shell
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup update                  # upgrade to the latest stable
rustup component add clippy rustfmt rust-analyzer
```

Go	Rust
`go build`	`cargo build` (`--release` for optimised)
`go run .`	`cargo run`
`go test ./...`	`cargo test`
`go vet`, staticcheck	`cargo clippy`
`gofmt` / `go fmt`	`cargo fmt` (rustfmt)
`go doc`, pkg.go.dev	`cargo doc --open`, docs.rs
`go mod init`	`cargo new` / `cargo init`
`go get pkg@v1`	`cargo add crate@1`
`go.mod` / `go.sum`	`Cargo.toml` / `Cargo.lock`
gopls	rust-analyzer
`GOOS=linux GOARCH=arm64`	`--target aarch64-unknown-linux-gnu`

Cargo is the build system and package manager. It calls `rustc`, the compiler, which you will rarely invoke directly.

One difference you'll notice straight away is compile time. Go was designed to compile fast; Rust was not. A clean build of a medium project

with dependencies can take a minute or more. Incremental debug builds are reasonable. Use `cargo check`, which type-checks without generating code, for the edit–compile loop, and let rust-analyzer run it in your editor.

> **COMING FROM GO**
>
> Go cross-compiles out of the box. Rust needs the target's standard library (`rustup target add aarch64-unknown-linux-gnu`) and often a linker for that platform. Tools like `cross` or `cargo zigbuild` hide most of this.

1.2 Hello, Cargo

SHELL
```
cargo new hello
cd hello
cargo run
```

This creates:

OUTPUT
```
hello/
├── Cargo.toml
└── src/
    └── main.rs
```

TOML
```
[package]
name = "hello"
version = "0.1.0"
edition = "2024"

[dependencies]
```

RUST
```
fn main() {
    println!("Hello, world!");
}
```

`println!` ends in `!` because it is a **macro**, not a function. Macros can take a variable number of arguments and check the format string at compile time, which a normal Rust function can't do. Format strings use `{}` where Go uses verbs:

Go	Rust
`fmt.Println(x)`	`println!("{x}")` or `println!("{}", x)`
`fmt.Printf("%v", x)`	`println!("{}", x)` (needs `Display`)
`fmt.Printf("%+v", x)`	`println!("{:?}", x)` (needs `Debug`)
`fmt.Printf("%#v", x)`	`println!("{:#?}", x)` (pretty `Debug`)
`fmt.Sprintf(...)`	`format!(...)`
`fmt.Fprintf(os.Stderr..)`	`eprintln!(...)`
`%5.2f`, `%-10s`, `%x`	`{:5.2}`, `{:<10}`, `{:x}`

Variables can be captured inline (`{name}`), but only simple identifiers can be captured this way, not expressions like `{user.name}`.

A **crate** is a compilation unit: one library or one binary, which is close to a Go module that contains a single package. A **package** in Cargo's sense is a `Cargo.toml` that builds one or more crates. `src/main.rs` is the root of a binary crate and `src/lib.rs` is the root of a library crate. A package may have both. Chapter 12 covers modules in detail.

1.3 Editions

Rust promises backward compatibility, like Go's Go 1 promise. Changes that would break code ship in **editions** (2015, 2018, 2021, 2024), chosen per crate in `Cargo.toml`. Crates on different editions link together without any problem. It plays a similar role to the `go` directive in `go.mod`, which also controls language semantics, such as the per-iteration loop variable change in Go 1.22.

1.4 Debug vs release

Debug builds (`cargo build`) are unoptimised and can be 10–50× slower than release builds, much more than the gap you'd see in Go. Integer overflow panics in debug and wraps in release. Always benchmark with `--release`.

```toml
[profile.release]
lto = true          # cross-crate inlining; slower link, faster binary
codegen-units = 1
strip = true
```

CHAPTER 2

The Basics, Quickly

This chapter covers the syntax you need before the interesting parts. It is deliberately brief.

2.1 Variables and mutability

GO

```go
x := 5
var y int = 10
x = 6 // fine
```

RUST

```rust
let x = 5;
let y: i32 = 10;
x = 6; // error: cannot assign twice to immutable variable
```

Bindings are **immutable by default**. You opt into mutation with `mut`:

RUST

```rust
let mut count = 0;
count += 1;
```

This isn't just style. `mut` is part of the ownership system you'll meet in Chapter 3: the compiler uses it to reason about who can change what.

Rust allows **shadowing**: declaring a new variable with the same name, often with a different type. It's idiomatic for transforming a value step by step:

RUST

```rust
let input = "42";
let input: i64 = input.parse().unwrap(); // new binding, new type
```

In Go, `:=` with the same name in an inner scope also shadows, and linters warn about it because it's usually a bug. In Rust, shadowing in the same scope is intentional and common.

Constants are written `const MAX: u32 = 100;` and always need a type. `static` declares a global with a fixed memory address; Chapter 13 shows how to make mutable globals safe.

2.2 Scalar types

Go	Rust
`int8` … `int64`	`i8` … `i64`, plus `i128`
`uint8` … `uint64`	`u8` … `u64`, plus `u128`
`int`, `uint`	`isize`, `usize` (pointer-sized)
`uintptr`	`usize`
`float32`, `float64`	`f32`, `f64`
`bool`	`bool`
`byte`	`u8`
`rune`	`char` (a Unicode scalar value, 4 bytes)

Two differences matter in practice.

Indexes and lengths are `usize`. Go uses `int` for lengths. Rust uses `usize`, and there are **no implicit conversions**, not even widening ones. You'll be writing `as` casts or `try_from` calls:

```rust
let n: i32 = 7;
let v = vec![0u8; 10];
let item = v[n as usize];          // `as` never fails: may truncate/wrap
let m = u16::try_from(70_000i32); // Err(...): checked conversion
```

Overflow is an error. In debug builds `255u8 + 1` panics; in release it wraps. When you mean wrapping, say so explicitly with `wrapping_add`, `checked_add` (returns `Option`), `saturating_add` or `overflowing_add`.

Integer literals take suffixes (`10u64`, `1_000_000i64`); `_` is a visual separator, as in Go.

2.3 Tuples and arrays

```rust
let pair: (i32, &str) = (1, "one");
let (n, name) = pair;          // destructuring
let first = pair.0;

let arr: [u8; 4] = [1, 2, 3, 4]; // fixed size, like Go's [4]byte
let zeros = [0; 256];
```

A tuple works like an anonymous struct. The empty tuple `()`, called **unit**, is what functions with no return value return. Go has no direct equivalent, but you can think of it as `struct{}`.

Arrays have fixed length and are values, just as in Go. Growable sequences (`Vec<T>`) and slices (`&[T]`) come in Chapter 9.

2.4 Functions

```rust
fn add(a: i32, b: i32) -> i32 {
    a + b // no semicolon: this is the return value
}

fn divmod(a: u32, b: u32) -> (u32, u32) {
    (a / b, a % b)
}
```

There are no named return values, no variadic functions (macros cover those uses), no default arguments and no function overloading. Multiple returns are written as a tuple. Functions may be declared after they're used, just like in Go.

2.5 Everything is an expression

This is the first real shift in thinking. In Rust, **blocks are expressions**. The last expression in a block, written without a trailing semicolon, is the block's value. Adding the semicolon turns it into a statement whose value is `()`.

```rust
let category = if score > 90 { "high" } else { "low" };

let parsed = {
    let raw = read_config();
    raw.trim().to_lowercase()
}; // `parsed` is the value of the block's final expression
```

`if` replaces the ternary operator Go doesn't have. `match` (Chapter 6) and even `loop` can produce values too:

```rust
let mut tries = 0;
let result = loop {
    tries += 1;
    if let Some(v) = attempt() {
        break v; // break with a value
    }
};
```

`return` exists for early exits. Ending a function on a bare expression is the idiomatic way to return its result.

> **WATCH OUT**
> A stray semicolon at the end of a function produces the error "mismatched types: expected `i32`, found `()`". Delete the semicolon.

2.6 Control flow

```rust
if x > 0 { /* braces required, no parens needed */ }

while n > 0 { n -= 1; }

loop { /* infinite, like Go's bare `for {}` */ }

for i in 0..10 { }          // 0 to 9, like `for i := range 10`
for i in 0..=10 { }         // inclusive range
for item in &items { }      // iterate by reference
for (i, item) in items.iter().enumerate() { }

'outer: for row in grid {
    for cell in row {
```

```
            if cell == 0 { continue 'outer; }
        }
    }
```

Go's single `for` keyword is split into `loop`, `while` and `for … in`. Rust's `for` always iterates over something that implements `IntoIterator`. There is no C-style three-clause loop; use a range.

`if` conditions must be `bool`. There is no truthiness, which matches Go.

2.7 Comments and docs

RUST

```
// line comment
/* block comment */

/// Doc comment for the item that follows. Markdown is supported.
/// Code blocks in doc comments are compiled and run as tests.
fn documented() {}

//! Doc comment for the enclosing module or crate (top of file).
```

In Go, any comment directly above a declaration becomes its documentation. In Rust, doc comments use their own syntax (`///`), and `cargo doc` renders them as HTML.

2.8 Naming conventions

Rust uses `snake_case` for functions, variables and modules, `CamelCase` for types and traits, and `SCREAMING_CASE` for constants. Unlike Go, capitalisation has **nothing** to do with visibility: items are private unless marked `pub`. rustc will warn if you break these conventions.

CHAPTER 3

Ownership

This chapter and the next two cover what makes Rust Rust. Nothing in Go prepares you for them directly, but you already understand the underlying problems. You just haven't had to state the solutions in your type signatures before.

3.1 The problem ownership solves

Every program that allocates memory has to decide when to free it. There are three classic answers:

1. **The programmer frees it** (C, C++). Fast and predictable, but use-after-free, double-free and leaks are routine bugs, and many of them become security vulnerabilities.
2. **A garbage collector frees it** (Go, Java). Safe and convenient. You pay in runtime overhead, unpredictable pauses (small in modern Go, but not zero), higher memory use, and the need for a runtime.
3. **The compiler works out when to free it** (Rust). Every value has a single *owner*, and when the owner goes out of scope the value is freed. The compiler checks this statically, so there's no runtime cost and no dangling pointers.

Option 3 is only possible if the compiler can always tell who owns what. That requirement is what makes the language feel different. Code that would be harmless in Go, like two variables pointing to the same heap data, needs to be spelled out in Rust.

3.2 Stack, heap and Go's escape analysis

In Go, you rarely think about the stack and the heap. You write `&T{}` anywhere, and escape analysis decides whether the value can live on the stack

or must be moved to the heap. If a pointer outlives its function, the value escapes and the GC takes care of it.

Rust makes this explicit. Local values live on the stack by default. Heap allocation happens through specific types: `Box<T>` for a single value, `Vec<T>` for a growable array, `String` for growable text, and a few others. These types are small handles on the stack (a pointer, plus length and capacity for `Vec` and `String`) that own a heap buffer. That layout is exactly the same as a Go slice header. The difference is what happens when you copy the handle.

3.3 The rules

1. Each value has exactly one **owner**, a variable or a field.
2. When the owner goes out of scope, the value is **dropped** (freed).
3. Ownership can be **moved** to a new owner. After a move, the old owner can no longer be used.

3.4 Moves

GO

```go
a := []int{1, 2, 3}
b := a        // copies the slice header; a and b share the array
b[0] = 99
fmt.Println(a[0]) // 99. Aliasing, which is sometimes surprising
```

RUST

```rust
let a = vec![1, 2, 3];
let b = a;          // ownership of the heap buffer MOVES to b
println!("{:?}", a); // error[E0382]: borrow of moved value: `a`
```

The bytes copied are the same in both languages: pointer, length and capacity. What differs is that Rust **invalidates the source**. If both `a` and `b` were usable, both would try to free the same buffer when they went out of scope (a double free), or one would see changes made by the other (aliasing). The move rule rules out both.

Passing an argument to a function is also a move:

```rust
fn consume(v: Vec<i32>) {
    println!("{}", v.len());
} // v dropped here, buffer freed

fn main() {
    let data = vec![1, 2, 3];
    consume(data);
    consume(data); // error: use of moved value
}
```

So is returning a value, which moves ownership out to the caller:

```rust
fn make() -> Vec<i32> {
    let v = vec![1, 2, 3];
    v // moved to the caller; NOT freed here
}
```

In Go you'd be returning a slice and the GC would keep the array alive. In Rust, the move tells the compiler that responsibility for freeing the buffer has passed to the caller.

> **KEY IDEA**
>
> A move is a shallow bitwise copy followed by the compiler forgetting the source. It costs no more than copying a Go slice header. Moves never copy heap data.

The compiler tracks moves along every control-flow path:

```rust
let s = String::from("hi");
if cond {
    drop(s); // moved into drop() on this path
}
println!("{s}"); // error: value used after being moved
                 // (on at least one path)
```

3.5 `Copy` types

Not everything moves. Integers, floats, `bool`, `char`, and tuples or arrays made only of these, are `Copy`: assigning them copies the bits and leaves the original usable, as you'd expect.

```rust
let x = 5;
let y = x;
println!("{x} {y}"); // fine: i32 is Copy
```

A type can be `Copy` only if a bitwise copy is a complete, independent duplicate. That means it owns no heap memory or other resources. Your own structs can opt in with `#[derive(Clone, Copy)]` (Chapter 8) when all their fields are `Copy`.

3.6 `Clone`: explicit deep copies

When you really do want two independent copies of heap data, ask for one:

```rust
let a = vec![1, 2, 3];
let b = a.clone(); // allocates a new buffer and copies elements
println!("{a:?} {b:?}");
```

Go has no single equivalent. You'd write `slices.Clone(a)`, `maps.Clone(m)`, or copy fields by hand, and a "deep copy" of a struct holding pointers isn't well defined. In Rust, `.clone()` is a method from the `Clone` trait, and each type defines what cloning means for it.

Because cloning is explicit, allocations are visible in the code. When you see `.clone()` in a hot loop, you know there's a cost there.

> **NOTE**
>
> Beginners often add `.clone()` until the compiler stops complaining. That's fine while you're learning: correct code that clones a little too much is better than code that doesn't compile. Once the ownership model feels familiar, most of those clones turn into borrows (next chapter).

3.7 Drop: deterministic destruction

When an owner goes out of scope, Rust calls the value's destructor right away, deterministically, in reverse order of declaration. Types customise this by implementing the `Drop` trait:

```rust
struct TempFile { path: String }

impl Drop for TempFile {
    fn drop(&mut self) {
        let _ = std::fs::remove_file(&self.path);
    }
}

fn work() {
    let tmp = TempFile { path: "/tmp/x".into() };
    // ... use tmp ...
} // tmp.drop() runs here, on every exit path, including panics
```

This pattern is known as **RAII** (Resource Acquisition Is Initialisation). It's how Rust closes files, releases locks, closes sockets and commits or rolls back transactions. There's usually no need for a `defer`-like construct:

```go
f, err := os.Open(path)
if err != nil { return err }
defer f.Close()
```

```rust
let f = File::open(path)?; // closed automatically when `f` is dropped
```

COMING FROM GO

`defer` is tied to the *function*, while `Drop` is tied to the *value*. That means:

- You can't forget to clean up. There's no `defer` to forget to write.
- Cleanup follows the value when it moves. Returning a `File`, or putting it in a struct, moves the responsibility for closing it along with it. A Go `defer f.Close()` in the wrong function closes the file too early.
- In a loop, each iteration's value is dropped at the end of that iteration. In Go, `defer` in a loop piles up until the function returns.

You can drop a value early by calling `drop(value)`. That's simply a function that takes ownership and does nothing, so the value is dropped when its parameter goes out of scope. You can't call `.drop()` directly.

There's one caveat: errors returned during cleanup are ignored, because `drop` can't return anything. For files, call `file.sync_all()?` explicitly if you need to know whether data reached disk. That's the same advice as checking the error from `Close` in Go.

3.8 Ownership is a tree

Owned values nest. A `Vec<String>` owns its buffer, and each `String` in it owns its own buffer. A struct owns its fields. When you drop the root, everything beneath it is dropped recursively. So a Rust program's data forms a **tree of ownership**, with each node having exactly one parent.

Go's object graph can be any shape: cycles, multiple parents, back pointers. The GC handles all of it. In Rust, any shape other than a tree needs explicit tools: borrowing (temporary access without ownership, Chapter 4), reference counting (`Rc` / `Arc`, Chapter 11) or indices into an arena. That's why a doubly linked list is a classic Rust headache, and why idiomatic Rust designs often look a little different from Go designs. Data flows down the tree, and shared access is temporary.

3.9 Moving out of things

You can move out of a local variable, but not out of something you only have access to through a reference, and not out of the middle of a `Vec`. Doing so would leave a hole:

```rust
let names = vec![String::from("a"), String::from("b")];
let first = names[0];          // error: cannot move out of index
let first = names[0].clone();  // ok: copy it
let first = &names[0];         // ok: borrow it (Chapter 4)
let first = names.into_iter().next(); // ok: consume the whole Vec
```

You can move fields out of an owned struct one at a time (a "partial move"). After that, the struct as a whole can't be used, though its other fields still can. When you want to take a value and leave something valid behind, use

`std::mem::take` (leaves `Default`), `std::mem::replace`, or `Option::take` (leaves `None`). These come up frequently when working with `&mut` references.

```rust
let mut buf = String::from("data");
let old = std::mem::take(&mut buf); // buf is now "", old is "data"
```

3.10 Summary

- Assignment, argument passing and return **move** non-`Copy` values. The source is unusable afterwards.
- Moves are cheap shallow copies. `.clone()` is the explicit, potentially expensive deep copy.
- Values are freed when their owner goes out of scope (`Drop`). That replaces both the GC and `defer`.
- Data forms an ownership tree. Sharing requires borrowing or explicit shared-ownership types.

CHAPTER 4

Borrowing

If moving were the only way to pass values around, every function would have to take ownership and hand it back. **Borrowing** lets you give code temporary access to a value without transferring ownership.

4.1 References

```rust
fn total_len(words: &Vec<String>) -> usize {
    let mut n = 0;
    for w in words {
        n += w.len();
    }
    n
}

let words = vec!["a".to_string(), "bc".to_string()];
let n = total_len(&words); // lend `words`
println!("{n} {words:?}"); // still ours
```

`&T` is a **shared reference** and `&mut T` is a **mutable (exclusive) reference**. At machine level both are just pointers, like Go's `*T`. They differ from Go pointers in three ways:

- **They never own.** Dropping a reference does nothing. The owner frees the value.
- **They're never null.** "Maybe there's a value" is written `Option<&T>` (Chapter 6).
- **They can never dangle.** The compiler proves that the referent outlives every reference to it. You can't return a reference to a local, keep a reference into a `Vec` that has since reallocated, or use a reference after the owner has been dropped.

```rust
fn dangle() -> &String {   // error: missing lifetime specifier
    let s = String::from("hi");
    &s                     // s is dropped at the end of this fn
}
```

In Go this would be fine, because escape analysis would move `s` to the heap. In Rust you return the owned `String` instead.

4.2 The borrowing rule

At any point in the program, a value can have **either**:

- any number of shared references `&T`, **or**
- exactly one mutable reference `&mut T`,

but never both. While a value is borrowed, its owner is also restricted. It can't move the value away, and if the borrow is shared, it can't mutate it.

> **KEY IDEA**
>
> **Aliasing XOR mutability.** Data can be shared or mutated, but not both at the same time. Nearly every borrow checker error comes back to this rule.

The following Go bug is exactly what the rule prevents:

```go
users := []User{{Name: "ann"}}
first := &users[0]
users = append(users, User{Name: "bob"}) // may reallocate
first.Name = "ANN" // writes to the OLD array; users[0] unchanged
```

Go doesn't crash here, because the GC keeps the old array alive, but the program silently does the wrong thing. In Rust:

```rust
let mut users = vec![User { name: "ann".into() }];
let first = &mut users[0];
users.push(User { name: "bob".into() });
// error[E0499]: cannot borrow `users` as mutable more than once
first.name = "ANN".into();
```

`first` is a live mutable borrow of `users`, and `push` needs another one. The same rule rules out iterator invalidation, modifying a map while ranging over it, and, as Chapter 13 shows, data races between threads.

4.3 Borrows end at last use

The borrow checker reasons about where each reference is *used*, not about lexical scope. A borrow lasts from its creation to its last use. (The feature is called *non-lexical lifetimes*.)

```rust
let mut v = vec![1, 2, 3];
let first = &v[0];
println!("{first}"); // last use of `first`
v.push(4);           // fine: the shared borrow has ended
```

Swapping the last two lines gives an error, because `first` would still be alive when `push` needs exclusive access.

4.4 Using references

You create references with `&` and `&mut`, and dereference them with `*`. Most of the time dereferencing is automatic. The method-call operator `.` dereferences as many times as needed, much as Go lets you call methods through a pointer:

```rust
let mut n = 10;
let r = &mut n;
*r += 1;               // explicit deref for assignment

let s = String::from("hi");
let rs = &s;
let rrs = &rs;
println!("{}", rrs.len()); // auto-deref through && to String
```

Comparison operators work through references (`&a == &b` compares the values). Arithmetic on `&i32` works for the common cases.

4.5 Methods and `self`

Methods live in `impl` blocks, and the receiver says how the method uses the value:

```rust
struct Counter { hits: u64 }

impl Counter {
    fn new() -> Self { Counter { hits: 0 } } // no receiver: a constructor
    fn get(&self) -> u64 { self.hits }        // shared borrow
    fn hit(&mut self) { self.hits += 1; }     // exclusive borrow
    fn finish(self) -> u64 { self.hits }      // takes ownership
}

let mut c = Counter::new();
c.hit();             // auto-borrows &mut c
let n = c.finish();  // c moved; can't be used after this
```

Go receiver	Rust receiver	Meaning
`func (c Counter)`	`self`	Go copies; Rust takes ownership
`func (c *Counter)`	`&self`	read-only access
`func (c *Counter)`	`&mut self`	read–write access

Go uses pointer receivers for two different reasons, to avoid copying and to allow mutation, and the signature doesn't say which. Rust's signature does. A `self` receiver is a *consuming* method: `finish` above, `into_iter()`, `String::into_bytes()`, or a builder's `build()`. These can express "after this call, the object is gone", which Go can't express.

4.6 Choosing parameter types

A practical rule for function signatures:

- Take `&T` when you only need to read. This is the most common case.
- Take `&mut T` when you need to modify the caller's value in place.
- Take `T` by value when you need to *keep* it, for example to store it in a struct or send it to another thread, or when it's small and `Copy`.

Accept the most general borrowed form. Use `&str` rather than `&String`, and `&[T]` rather than `&Vec<T>`. Both of these coerce automatically (Chapter 9):

```rust
fn shout(s: &str) -> String { s.to_uppercase() }
fn sum(xs: &[i64]) -> i64 { xs.iter().sum() }

let owned = String::from("hey");
shout(&owned);   // &String coerces to &str
shout("literal");
sum(&vec![1, 2]);
sum(&[1, 2, 3]);
```

4.7 Fighting the borrow checker

The borrow checker is conservative. Sometimes it rejects programs that are actually safe. Learning Rust largely means learning a few standard ways to restructure code. These are the situations you'll run into in your first weeks.

1. Mutating a collection while iterating

```rust
for item in &items {
    if item.expired() {
        items.retain(|i| !i.expired()); // error: items already borrowed
    }
}
```

Look for a method that does the job in one go (`retain`, `drain`, `dedup_by`, `sort_by_key`). Otherwise, collect what you need first and mutate afterwards, or iterate over indices:

```rust
items.retain(|i| !i.expired());

for i in 0..items.len() {
    if items[i].needs_bump() { items[i].bump(); }
}
```

2. Borrowing two fields through methods

```rust
impl Game {
    fn player_mut(&mut self) -> &mut Player { &mut self.player }

    fn tick(&mut self) {
```

```
        let p = self.player_mut();  // borrows ALL of self mutably
        p.update(&self.world);      // error: self already borrowed
    }
}
```

The compiler tracks borrows of individual fields within a function body, but a method call borrows the whole receiver. Access the fields directly so that the borrows are visibly disjoint:

```rust
fn tick(&mut self) {
    let p = &mut self.player;
    p.update(&self.world); // fine: different fields
}
```

For two disjoint elements of a slice, use `split_at_mut`, or `get_disjoint_mut` on slices and `HashMap`s.

3. Look up, then insert

```rust
fn get_or_compute(map: &mut HashMap<String, u64>, key: String)
    -> &u64
{
    if let Some(v) = map.get(&key) {
        return v; // returning the borrow keeps `map` borrowed...
    }
    map.insert(key.clone(), compute()); // ...so this is an error
    &map[&key]
}
```

This is a known limitation of the current borrow checker: the code is safe, and a future checker (Polonius) will accept it. Today you use the **entry API**, which also does a single hash lookup instead of two or three:

```rust
fn get_or_compute(map: &mut HashMap<String, u64>, key: String)
    -> &u64
{
    map.entry(key).or_insert_with(compute)
}
```

4. Storing a reference in a long-lived struct

Usually you want the struct to own the data instead. See the next chapter.

> **COMING FROM GO**
>
> You might expect the borrow checker to feel like a stricter `go vet`. In practice it pushes your design towards data with a single clear owner, mutation in few places, and short-lived shared access. These are good habits in Go too, especially in concurrent code. Most experienced Rust programmers say that after a few months they rarely fight the borrow checker, because they've stopped writing the shapes it rejects.

4.8 Slices are borrows

A slice `&[T]` is a borrowed view into contiguous memory (pointer + length), just like Go's slice header but without the capacity and without the ability to grow. `&mut [T]` lets you modify elements, but not change the length.

RUST

```
let mut v = vec![5, 3, 8, 1];
let middle = &v[1..3];        // [3, 8]
let tail = &mut v[2..];
tail.sort();                  // sorts only the tail
```

Because a slice borrows from its owner, the owner can't reallocate while the slice exists. Go's `append` aliasing bugs, where two slices share an array and an `append` through one overwrites data visible to the other, can't happen in Rust.

4.9 Summary

- `&T` shares, `&mut T` grants exclusive access. Neither owns, and neither can dangle or be null.
- Shared XOR mutable. Borrows end at their last use.
- Receivers state their intent: `&self`, `&mut self`, or `self` (consume).
- When the checker refuses, restructure: use collection methods, access fields directly, use the entry API, or have structs own their data.

CHAPTER 5

Lifetimes

Every reference has a **lifetime**: the part of the program during which it's valid. Until now the compiler has worked these out on its own. Some signatures are ambiguous, though, and then you have to state the relationship yourself. This is the most unusual-looking syntax in Rust, but the idea behind it is simple.

5.1 Why the compiler needs help

```rust
fn longest(a: &str, b: &str) -> &str {
    if a.len() >= b.len() { a } else { b }
}
// error[E0106]: missing lifetime specifier
// help: this function's return type contains a borrowed value, but
//       the signature does not say whether it is borrowed from `a`
//       or `b`
```

The compiler checks each function on its own, looking only at the signatures of the functions it calls, never their bodies. A caller of `longest` needs to know how long the returned reference stays valid, and that depends on where it came from. Lifetime parameters say so:

```rust
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}
```

Read `'a` as "some region of code". The signature says that for some region `'a` where both inputs are valid, the output is valid for that same region. At each call site the compiler picks the largest `'a` that works, which is the overlap of the two arguments' lifetimes, and checks that the result isn't used beyond it:

```rust
let outer = String::from("long string");
let result;
{
    let inner = String::from("short");
    result = longest(&outer, &inner);
} // inner dropped here
println!("{result}");
// error[E0597]: `inner` does not live long enough
```

KEY IDEA

Lifetime annotations **never change** how long anything lives. They are constraints that link the lifetimes of inputs and outputs, so the compiler can check callers without looking inside the function. They're similar to generic type parameters, except that they describe validity instead of type.

If the output depends on only one input, say so, and the other argument is left unconstrained:

```rust
fn first_word<'a>(s: &'a str, _sep: &str) -> &'a str {
    s.split_whitespace().next().unwrap_or("")
}
```

5.2 Elision: why you rarely write them

Most signatures need no annotations because of three **elision rules**:

1. Each reference parameter gets its own lifetime.
2. If there is exactly one input lifetime, it is assigned to all output references.
3. If there is a `&self` or `&mut self` parameter, its lifetime is assigned to all output references.

So `fn trim(s: &str) -> &str` is shorthand for `fn trim<'a>(s: &'a str) -> &'a str`, and a method like `fn name(&self) -> &str` returns something borrowed from `self`. You only write lifetimes when the rules can't decide, which typically means several reference inputs and a reference output, or a struct that holds a reference.

5.3 Structs that hold references

A struct can borrow data instead of owning it, but it then has to carry a lifetime parameter. That way the compiler can make sure the struct doesn't outlive the data it borrows:

```rust
struct Parser<'src> {
    input: &'src str,
    pos: usize,
}

impl<'src> Parser<'src> {
    fn new(input: &'src str) -> Self {
        Parser { input, pos: 0 }
    }

    /// Returned tokens borrow from the *input*, not from the parser,
    /// so they stay valid after the parser is dropped.
    fn next_token(&mut self) -> Option<&'src str> {
        let rest = &self.input[self.pos..];
        let rest_trim = rest.trim_start();
        let start = self.pos + (rest.len() - rest_trim.len());
        let len = rest_trim.find(' ').unwrap_or(rest_trim.len());
        if len == 0 { return None; }
        self.pos = start + len;
        Some(&self.input[start..start + len])
    }
}
```

The return type is the interesting part. Under elision rule 3, the signature would have been `fn next_token(&mut self) -> Option<&str>`, tied to the borrow of `self`. Callers then couldn't keep a token while calling `next_token` again. Writing `'src` explicitly says that the token borrows from the source text, not from the parser. Choosing between these is the main design decision you make with lifetimes.

This is how Rust does **zero-copy parsing**. Tokens are slices of the original buffer and are checked so that they can't outlive it. Go does the same with `[]byte` subslices, but nothing stops you from holding a subslice after the buffer has been reused, for example with a `bufio.Scanner`, whose `Bytes()` documentation warns that the slice may be overwritten by the next call.

> **WATCH OUT**
>
> Don't reach for struct lifetimes by default. A struct holding a reference can't outlive whatever it borrows from, and every struct that contains it inherits the lifetime parameter. For application-level types such as config, request handlers or domain objects, **own your data** (`String`, `Vec<T>`) unless profiling says otherwise. Borrowing structs suit short-lived views: parsers, iterators, guards and formatters.

5.4 `'static`

`'static` is the lifetime of the entire program. String literals are `&'static str` because they live in the binary's read-only data.

You'll mostly see `'static` as a **bound**, `T: 'static`, and this is often misread. It does **not** mean that the value lives forever. It means that the type **contains no borrowed references with a shorter lifetime**. Every owned type, such as `String`, `Vec<u8>` or `Arc<Config>`, satisfies `'static`.

```rust
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T + Send + 'static,
    T: Send + 'static,
```

This is `std::thread::spawn`. The closure must be `'static` because the new thread may outlive the caller's stack frame, so it can't borrow local variables. In Go, a goroutine can capture anything because the GC keeps it alive. In Rust you move owned data in, share it through `Arc` (Chapter 11), or use scoped threads (Chapter 13).

5.5 Lifetimes in practice

A few more forms you'll see:

```rust
impl<'a> Parser<'a> { }        // impl for a lifetime-generic type
fn show(p: &Parser<'_>) { }     // '_ : "some lifetime, inferred"
struct Ctx<'a, T: 'a> { x: &'a T } // T must outlive 'a (usually implied)
fn f<'a, 'b: 'a>(x: &'a str, y: &'b str) { } // 'b outlives 'a
```

When you see an error about lifetimes, it almost always falls into one of these categories:

- **“does not live long enough”**: a reference is used after its owner is dropped. Move the owner to an outer scope, or return an owned value.
- **“lifetime may not live long enough”** or **“explicit lifetime required”**: a signature doesn’t say where a returned reference comes from. Add a lifetime parameter that links output to input.
- **“borrowed value must be valid for the static lifetime”** or **“may outlive borrowed value”**: you passed a borrow to a thread or task. Move owned data in instead (`move` closures, `Arc`, `clone`).

5.6 Summary

- A lifetime is the region in which a reference is valid. Annotations link regions together; they never extend them.
- The elision rules cover most functions. Annotate when a function returns a reference and has several reference inputs.
- Structs with references need lifetime parameters. Prefer owned fields unless you’re writing a short-lived view.
- `T: 'static` means “no short-lived borrows inside”, which is what threads and async tasks require.

CHAPTER 6

Structs, Enums and Pattern Matching

6.1 Structs

Structs look much like Go's:

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct User {
    pub id: u64,
    pub name: String,
    email: Option<String>, // private field
}

let name = String::from("ann");
let u = User { id: 1, name, email: None }; // field init shorthand
let v = User { id: 2, ..u.clone() };       // struct update syntax
```

There are a few variants:

```rust
struct Meters(f64);          // tuple struct; access with .0
struct Point3(i32, i32, i32);
struct Marker;               // unit struct, zero-sized
```

A single-field tuple struct is the **newtype** pattern, the counterpart of Go's `type UserID int64`. It creates a distinct type with no runtime cost. Unlike in Go, a newtype doesn't inherit any methods or operators from the wrapped type. You add what you need, usually through `derive`.

No zero values

This is a significant difference. Go initialises every variable to a zero value, so `var u User` is always valid, and "the zero value should be useful" is a design goal. **Rust has no zero values.** Every field must be initialised explicitly, and reading an uninitialised variable is a compile error.

The upside is that a type can't end up half-constructed. A `User` always has a name, and a `Mutex` is never accidentally copied in its zero state. When a sensible default exists, the type implements the `Default` trait:

RUST

```rust
#[derive(Default)]
struct Options {
    retries: u32,       // 0
    verbose: bool,      // false
    tag: String,        // ""
}

let o = Options { retries: 3, ..Default::default() };
```

Constructors are plain associated functions, by convention named `new`, or `with_…` / `from_…` for variants. Unlike Go's `NewUser` functions, they're defined in the type's `impl` block and called as `User::new(…)`. Because private fields can only be set inside the module, a private field plus a constructor gives you a guaranteed-valid type.

6.2 Enums are sum types

Go's `iota` constants are just named integers:

GO

```go
type State int
const (
    Idle State = iota
    Running
    Done
)
```

Rust enums can be used the same way, but each variant can also **carry its own data**:

RUST

```rust
enum Shape {
    Circle { radius: f64 },
    Rect { w: f64, h: f64 },
    Triangle(f64, f64, f64),
    Empty,
}
```

A value of type `Shape` is exactly one of these variants. The compiler stores a small tag recording which variant it is, alongside room for the largest variant's data. It isn't a pointer to an object on the heap.

In Go you'd model this with an interface and a type switch:

```go
type Shape interface{ isShape() }
type Circle struct{ Radius float64 }
type Rect struct{ W, H float64 }
func (Circle) isShape() {}
func (Rect) isShape()   {}

switch s := shape.(type) {
case Circle: ...
case Rect: ...
} // forgot a case? the compiler doesn't care
```

The Rust version is checked for **exhaustiveness**:

```rust
fn area(s: &Shape) -> f64 {
    match s {
        Shape::Circle { radius } => 3.14159 * radius * radius,
        Shape::Rect { w, h } => w * h,
        Shape::Triangle(a, b, c) => {
            let p = (a + b + c) / 2.0;
            (p * (p - a) * (p - b) * (p - c)).sqrt()
        }
        Shape::Empty => 0.0,
    }
}
```

Add a variant and every `match` that doesn't handle it stops compiling. This turns out to be one of Rust's most useful features. You can make a large refactor, follow the compiler errors, and when they're gone the change is complete.

COMING FROM GO

Enums can't be extended outside their crate, while interfaces are open to anyone. That's the trade-off. Use an enum when you have a **closed set of variants**: states, AST nodes, messages, results. Use a trait (Chapter 8) when you want an **open set** that others can extend. Go only gives you the open option, so you emulate closed sets with unexported marker methods and hope everyone writes a `default:` case.

Enums can have methods, just like structs:

RUST

```rust
impl Shape {
    fn is_empty(&self) -> bool { matches!(self, Shape::Empty) }
}
```

Make illegal states unrepresentable

A common Go struct:

GO

```go
type Conn struct {
    State     string    // "disconnected", "connecting", "connected"
    Addr      string    // set when connecting or connected
    SessionID string    // only valid when connected
    Retries   int       // only meaningful when connecting
}
```

Nothing stops `SessionID` from being set while disconnected. In Rust you put each piece of data in the state it belongs to:

RUST

```rust
enum Conn {
    Disconnected,
    Connecting { addr: String, retries: u32 },
    Connected { addr: String, session_id: String },
}
```

Now it's impossible to read a session ID from a connection that doesn't have one, and nothing needs to be checked at runtime.

6.3 Pattern matching

`match` is much more capable than Go's `switch`. Patterns can destructure nested data, bind variables, match ranges and have guards:

```rust
match msg {
    Msg::Move { x: 0, y } => println!("on the y axis at {y}"),
    Msg::Move { x, y } if x == y => println!("diagonal"),
    Msg::Move { .. } => println!("somewhere else"),
    Msg::Write(text) => println!("{text}"),
    Msg::Color(r, _, _) if r > 200 => println!("reddish"),
    Msg::Color(..) => {}
    Msg::Quit => return,
}

match code {
    200..=299 => "ok",
    301 | 302 | 307 | 308 => "redirect",
    n @ 400..=499 => { log(n); "client error" }
    _ => "other",
}
```

`match` is an expression, all arms must have the same type, and there is no fallthrough, so there's no `break` to forget and no `fallthrough` keyword.

Patterns also work in `let`, function parameters and `for` loops:

```rust
let (q, r) = divmod(17, 5);
let User { name, id, .. } = user;   // destructure a struct
for (k, v) in &map { }
fn dist(&(x, y): &(f64, f64)) -> f64 { (x * x + y * y).sqrt() }
```

When matching on a reference, bindings automatically become references too. Matching `&Shape` gives you `radius: &f64`. This "match ergonomics" feature is why the `area` function above works without any `&` or `*`.

Shorthand forms

```rust
if let Shape::Circle { radius } = s {
    println!("circle of radius {radius}");
}

// let-else: bind or bail out. Great for early returns.
```

```
let Some(user) = find_user(id) else {
    return Err(Error::NotFound);
};

while let Some(job) = queue.pop() { run(job); }

let is_big = matches!(s, Shape::Rect { w, .. } if *w > 100.0);
```

`let … else` corresponds to Go's guard clause, `if x == nil { return }`, except that afterwards `user` is a plain, unwrapped value.

Conditions can be chained with `&&` inside `if let` (stable since Rust 1.88, in the 2024 edition):

```
if let Some(u) = user && u.is_admin() && let Some(t) = u.token() {
    authorize(t);
}
```

RUST

6.4 `Option`: no more nil

Rust has no null. The possibility of a missing value is part of the type:

```
enum Option<T> {
    Some(T),
    None,
}
```

RUST

`Option` is an ordinary enum from the standard library. `Some` and `None` are in scope everywhere. A function that might not find something returns `Option<T>`, and the caller can't use the result without handling `None`:

```
fn find(users: &[User], id: u64) -> Option<&User> {
    users.iter().find(|u| u.id == id)
}

match find(&users, 7) {
    Some(u) => println!("{}", u.name),
    None => println!("no such user"),
}
```

RUST

Compare this with Go, where a `*User` may or may not be nil and only the documentation tells you which, or where you return `(User, bool)`. `Option` also removes the need for Go's "comma ok" idiom, since `map.get(&k)` returns an `Option<&V>`.

`Option` comes with a large set of combinators. These are the ones you'll use daily:

```rust
let name: Option<&str> = user.map(|u| u.name.as_str());
let name = name.unwrap_or("anonymous");
let len = maybe_s.as_deref().map_or(0, str::len);
let port: u16 = env_port.and_then(|p| p.parse().ok()).unwrap_or(8080);
let user = find(&users, id).ok_or(Error::NotFound)?; // into Result
let x = slot.take();       // move out, leave None behind
let v = cache.get_or_insert_with(compute);
```

`unwrap()` extracts the value and **panics** on `None`. `expect("msg")` does the same with a message. Use them in tests, in prototypes, and where `None` would be a bug, i.e. an invariant you've already established. In library code, propagate instead (next chapter).

> **NOTE**
>
> `Option<&T>`, `Option<Box<T>>` and `Option<NonZeroU32>` are the same size as the bare pointer or integer. The compiler uses the forbidden null (or zero) value to represent `None`. Using `Option` adds no overhead compared with a nullable pointer.

CHAPTER 7

Error Handling

Go and Rust agree on the basic philosophy: errors are values, returned from functions and handled explicitly, with no exceptions. Rust adds type-system support that removes most of the boilerplate and makes errors impossible to ignore.

7.1 `Result`

RUST

```rust
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

GO

```go
func parsePort(s string) (int, error) {
    n, err := strconv.Atoi(s)
    if err != nil {
        return 0, fmt.Errorf("parse port %q: %w", s, err)
    }
    return n, nil
}
```

RUST

```rust
fn parse_port(s: &str) -> Result<u16, std::num::ParseIntError> {
    let n = s.parse::<u16>()?;
    Ok(n)
}
```

A Go function returns a value **and** an error, and by convention you only look at the value when `err == nil`. That convention isn't enforced: nothing stops you from using `n` without checking `err`, or from using a garbage value. A Rust function returns a value **or** an error, and the only way to

reach the `T` inside a `Result` is to handle the `Err` case, whether by matching, propagating with `?`, or explicitly unwrapping.

`Result` is marked `#[must_use]`, so ignoring a returned `Result` produces a warning. To discard one deliberately, write `let _ = f();`.

7.2 The ? operator

`?` does the `if err != nil { return err }` for you:

```rust
fn read_config(path: &str) -> Result<Config, Error> {
    let text = std::fs::read_to_string(path)?;
    let cfg: Config = toml::from_str(&text)?;
    cfg.validate()?;
    Ok(cfg)
}
```

`expr?` evaluates to the `Ok` value, or returns early from the function with the `Err` value **converted** through the `From` trait into the function's error type. That conversion is why the two different error types above, `io::Error` and `toml::de::Error`, can both be propagated as `Error`. It works as long as `Error` implements `From<io::Error>` and `From<toml::de::Error>`.

`?` also works on `Option` inside functions that return `Option`: on `None` it returns `None` early.

> **COMING FROM GO**
>
> `?` is the boilerplate-free error propagation Go has debated for years. Rust keeps propagation **visible**: every `?` is a point where the function may return early, just as every `if err != nil` is in Go. What's missing compared with idiomatic Go is the *context*, the `"parse port %q: %w"` part. The next sections cover how to add it.

7.3 Defining error types

For a library, define an enum of the ways things can fail:

```rust
use std::fmt;

#[derive(Debug)]
```

```rust
pub enum ConfigError {
    Io(std::io::Error),
    Parse { line: usize, msg: String },
    MissingField(&'static str),
}

impl fmt::Display for ConfigError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Self::Io(e) => write!(f, "reading config: {e}"),
            Self::Parse { line, msg } => {
                write!(f, "line {line}: {msg}")
            }
            Self::MissingField(k) => write!(f, "missing field {k}"),
        }
    }
}

impl std::error::Error for ConfigError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            Self::Io(e) => Some(e),
            _ => None,
        }
    }
}

impl From<std::io::Error> for ConfigError {
    fn from(e: std::io::Error) -> Self { Self::Io(e) }
}
```

The pieces map onto Go as follows:

Go	Rust
`error` interface (`Error() string`)	`std::error::Error` trait + `Display`
`var ErrNotFound = errors.New(..)`	an enum variant
`errors.Is(err, ErrNotFound)`	`matches!(err, MyError::NotFound)`
`errors.As(err, &target)`	`match`, or `err.downcast_ref::<T>()`
`fmt.Errorf("..: %w", err)`	a variant wrapping the source error
`errors.Unwrap(err)`	`err.source()`

Callers handle specific errors with an ordinary `match`, and the compiler checks that every variant is covered:

```rust
match load("app.toml") {
    Ok(cfg) => run(cfg),
    Err(ConfigError::Io(e)) if e.kind() == ErrorKind::NotFound => {
        run(Config::default())
    }
    Err(e) => return Err(e.into()),
}
```

Compared with sentinel errors and `errors.Is` chains, the possible errors are now visible in the function's signature.

7.4 `thiserror` and `anyhow`

Writing those impls by hand gets tedious. In practice, the ecosystem has settled on two crates:

`thiserror` derives the boilerplate for library error types:

```rust
#[derive(Debug, thiserror::Error)]
pub enum ConfigError {
    #[error("reading config: {0}")]
    Io(#[from] std::io::Error),
    #[error("line {line}: {msg}")]
    Parse { line: usize, msg: String },
    #[error("missing field {0}")]
    MissingField(&'static str),
}
```

`anyhow` provides a single catch-all error type for applications, where you mostly want to propagate errors with context and report them at the top level:

```rust
use anyhow::{Context, Result, bail};

fn load_user(id: u64) -> Result<User> {
    let path = format!("users/{id}.json");
    let data = std::fs::read(&path)
        .with_context(|| format!("reading {path}"))?;
    let user: User = serde_json::from_slice(&data)
```

```
        .context("decoding user")?;
    if user.disabled {
        bail!("user {id} is disabled");
    }
    Ok(user)
}

fn main() -> Result<()> {
    let u = load_user(42)?;
    println!("{}", u.name);
    Ok(())
}
// Error: reading users/42.json
//
// Caused by:
//     No such file or directory (os error 2)
```

`.context(...)` corresponds to `fmt.Errorf("decoding user: %w", err)`, and the chain is kept so that you can walk or downcast it. `anyhow::Error` works like Go's `error` interface: a boxed, type-erased error that any error type converts into through `?`.

The rule of thumb is **`thiserror` in libraries** (callers need to match on specific failures) and **`anyhow` in binaries** (you mostly report errors). The standard library's `Box<dyn std::error::Error + Send + Sync>` is the no-dependency version of `anyhow::Error`.

7.5 Panics

`panic!` in Rust corresponds to `panic` in Go. It's for bugs and broken invariants, not for expected failures. Indexing out of bounds, `unwrap()` on `None`, integer overflow in debug builds and explicit `panic!("...")` calls all panic.

By default a panic unwinds the stack of the current thread, running `Drop` for everything on the way. If it reaches the top of the main thread, the process exits. A panic in another thread kills only that thread, and `join()` returns it as an `Err`. `std::panic::catch_unwind` exists, playing roughly the role of `recover`, but it's meant for FFI boundaries and thread pools, not for control flow. Setting `panic = "abort"` in the Cargo profile makes panics abort the process immediately, which produces smaller binaries.

> **COMING FROM GO**
>
> In Go, HTTP servers commonly `recover` panics per request. Rust web frameworks run each request as a task, and a panicking task is isolated by the runtime (Tokio catches it and reports it through the task's `JoinHandle`). Even so, panics are less common in Rust than in Go: there's no nil to dereference, and invalid states are often impossible to construct.

7.6 Useful patterns

RUST

```rust
// Collect an iterator of Results: stops at the first error.
let nums: Result<Vec<i32>, _> =
    lines.iter().map(|l| l.parse::<i32>()).collect();

// Option <-> Result
let v = opt.ok_or(MyError::Missing)?;
let o: Option<i32> = res.ok();

// Transform errors
let n = s.parse::<u16>().map_err(|e| ConfigError::Parse {
    line: 1,
    msg: e.to_string(),
})?;

// Fallbacks
let port = env_port().unwrap_or(8080);
let cfg = load().unwrap_or_else(|_| Config::default());
let cfg = load().unwrap_or_default();
```

7.7 Summary

- `Result<T, E>` returns a value *or* an error. You can't touch the value without dealing with the error.
- `?` propagates errors and converts them with `From`.
- Libraries define error enums (`thiserror`). Applications use `anyhow` and add context with `.context()`.
- `panic!` is for bugs, as in Go, but you'll need it much less often.

CHAPTER 8

Traits and Generics

Traits are Rust's version of interfaces, and they do considerably more: they constrain generics, provide operator overloading, drive `derive`, and mark thread-safety. This is the second largest conceptual step after ownership.

8.1 Defining and implementing traits

```rust
pub trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> String {          // default method
        String::from("shape")
    }
}

struct Square(f64);

impl Shape for Square {
    fn area(&self) -> f64 { self.0 * self.0 }
    // name() uses the default
}
```

Nominal, not structural

The first difference from Go: implementation is **explicit**. In Go, a type satisfies an interface just by having the right methods. In Rust, you write `impl Trait for Type`, and methods in a type's own `impl` block are not trait methods, even if their names and signatures match.

There are trade-offs both ways. Go's structural typing lets you define an interface in the consumer package after the fact, which Rust can't do. In return, Rust lets you implement a trait for a type you didn't write:

```rust
trait Shout { fn shout(&self) -> String; }

impl Shout for str {
    fn shout(&self) -> String { self.to_uppercase() + "!" }
}

"hello".shout(); // "HELLO!"
```

That's impossible in Go, where you can only declare methods on types in your own package. The limit in Rust is the **orphan rule**: you can implement a trait for a type only if your crate defines either the trait or the type. It stops two crates from providing conflicting implementations. To implement a foreign trait for a foreign type, wrap the type in a newtype.

Calling a trait method requires the trait to be **in scope**. If the compiler says "no method named `write_all` found", you probably need `use std::io::Write;`.

8.2 Generics

```rust
fn largest<T: PartialOrd>(items: &[T]) -> Option<&T> {
    let mut best = items.first()?;
    for it in items {
        if it > best { best = it; }
    }
    Some(best)
}
```

```go
func Largest[T cmp.Ordered](items []T) (T, bool) { ... }
```

They look similar, but there are important differences:

Go generics	Rust generics
constraints are interfaces/type sets	bounds are traits
`~int | ~float64` type sets	no union types; use a trait
no generic methods	generic methods are fine
GC-shape stenciling (partly boxed)	full monomorphisation
operators via `cmp.Ordered` etc.	operators via traits (`Add`, `PartialOrd`)

Go generics	Rust generics
type inference: function args	inference across the whole body

Monomorphisation means the compiler generates a separate copy of `largest` for each concrete `T` it's used with, and optimises each copy as if you'd written it by hand. Generic Rust code is therefore as fast as specialised code. You pay in compile time and binary size. Go's implementation shares one copy among all pointer types and passes a dictionary, which compiles faster but can be slower at runtime.

When bounds grow, a `where` clause keeps the signature readable:

```rust
fn summarize<K, V>(map: &HashMap<K, V>) -> String
where
    K: Display + Eq + Hash,
    V: Debug,
{
    // ...
}
```

Generic types work as you'd expect:

```rust
struct Stack<T> { items: Vec<T> }

impl<T> Stack<T> {
    fn push(&mut self, t: T) { self.items.push(t); }
    fn pop(&mut self) -> Option<T> { self.items.pop() }
}

// Methods that exist only when T meets extra bounds:
impl<T: Display> Stack<T> {
    fn render(&self) -> String {
        self.items.iter().map(|t| t.to_string())
            .collect::<Vec<_>>().join(", ")
    }
}
```

The second `impl` block, called a *conditional impl*, has no equivalent in Go.

8.3 Static vs dynamic dispatch

A Go interface value is always dynamic. It's a two-word pair of a type descriptor (itab) and a data pointer, and calls go through a method table. Rust lets you choose.

Static dispatch uses generics or `impl Trait`:

```rust
fn print_area(s: &impl Shape) {   // sugar for <S: Shape>(s: &S)
    println!("{}", s.area());
}

fn make_shape() -> impl Shape {    // "some type that implements Shape"
    Square(2.0)
}
```

`impl Trait` in return position hides the concrete type from the caller. It's essential for returning closures and iterators, whose types can't be written down. The function must still return **one** concrete type.

Dynamic dispatch uses **trait objects**, written `dyn Trait`:

```rust
let shapes: Vec<Box<dyn Shape>> = vec![
    Box::new(Square(1.0)),
    Box::new(Circle(2.0)),
];
for s in &shapes {
    println!("{}: {}", s.name(), s.area()); // vtable call
}
```

`dyn Shape` has no fixed size, so it always lives behind a pointer: `&dyn Shape`, `Box<dyn Shape>` or `Arc<dyn Shape>`. Such a pointer is a **fat pointer** (data pointer plus vtable pointer), the same layout as a Go interface value.

> **COMING FROM GO**
>
> `Box<dyn Trait>` is the closest thing to a Go interface value, and the right tool when you need a heterogeneous collection or a plugin point chosen at runtime. Most Rust code uses generics by default and switches to `dyn` when it needs that runtime flexibility, or when monomorphisation would bloat the binary.

Not every trait can be used as `dyn`. It must be **dyn-compatible** (formerly "object safe"): methods can't be generic, and they can't return `Self` by value. The reason is that the vtable has to be built ahead of time, which isn't possible for an unbounded family of generic methods. The compiler will tell you when you hit this.

Dynamic downcasting, Go's `v.(Concrete)`, is available through `std::any::Any`, but it's rarely needed. If you find yourself wanting it, an enum is usually the better design.

8.4 Associated types and constants

A trait can declare types and constants that each implementation fills in:

```rust
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // ...plus ~75 provided methods: map, filter, sum, ...
}

struct Countdown(u32);

impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) -> Option<u32> {
        if self.0 == 0 { return None; }
        self.0 -= 1;
        Some(self.0 + 1)
    }
}
```

The difference from a generic parameter (`trait Iterator<T>`) is that an associated type is chosen once per implementation. `Countdown` is an iterator of `u32`, period. Bounds can constrain it: `fn total<I: Iterator<Item = u32>>(it: I)`.

8.5 Supertraits and blanket impls

```rust
trait Entity: Debug + Clone {   // implementors must also be Debug+Clone
    fn id(&self) -> u64;
}
```

```
// Blanket impl: every type that implements Display gets ToString.
// (This is literally how std defines `to_string()`.)
impl<T: Display + ?Sized> ToString for T {
    fn to_string(&self) -> String { /* ... */ }
}
```

`?Sized` relaxes the implicit requirement that `T` has a known size, so the impl also covers `str` and `dyn Trait`.

8.6 The standard traits you need to know

Most of your types will `#[derive(...)]` a handful of these. A derive is compiler-generated boilerplate, field by field.

Trait	Purpose	Go analogue
`Debug`	`{:?}` formatting	`%+v` (automatic in Go)
`Display`	`{}` formatting, user-facing	`String() string`
`Clone`	explicit `.clone()`	manual copying
`Copy`	implicit bitwise copy	value semantics
`PartialEq`, `Eq`	`==`	`comparable`
`PartialOrd`, `Ord`	`<`, sorting	`cmp.Ordered`, `cmp.Compare`
`Hash`	usable as a map key	`comparable`
`Default`	`T::default()`	zero value
`From`, `Into`	infallible conversion	conversion functions
`TryFrom`, `TryInto`	fallible conversion	functions returning error
`FromStr`	`"42".parse::<T>()`	`strconv.ParseX`
`AsRef`, `Borrow`	cheap reference conversion	—
`Deref`	`*x`, auto-deref to inner type	embedding (loosely)
`Drop`	destructor	`defer` / finalizers
`Iterator`, `IntoIterator`	`for` loops	range-over-func
`Send`, `Sync`	thread safety (Chapter 13)	—

A few notes:

- `PartialEq` vs `Eq` (and `PartialOrd` vs `Ord`) exists because of floats. `NaN != NaN`, so `f64` is only `PartialEq`. `HashMap` keys need `Eq + Hash`, which is why you can't key a map by `f64`. Go lets you do that and quietly gets it wrong for NaN.
- Implement `From<A> for B` and you get `Into<B> for A` for free. Accepting `impl Into<String>` lets callers pass either `&str` or `String`.
- Go prints any value with `%v` through reflection. Rust has no runtime reflection, so a type is printable only if it implements `Debug` or `Display`. Derive `Debug` on almost everything.

Operator overloading

Operators are traits in `std::ops`:

```rust
use std::ops::Add;

#[derive(Debug, Clone, Copy, PartialEq)]
struct V2 { x: f64, y: f64 }

impl Add for V2 {
    type Output = V2;
    fn add(self, o: V2) -> V2 { V2 { x: self.x + o.x, y: self.y + o.y } }
}

let c = V2 { x: 1.0, y: 2.0 } + V2 { x: 0.5, y: 0.5 };
```

Go deliberately doesn't support this. Rust supports it but keeps it restrained: there are no custom operators, and the set of overloadable operators is fixed (`Add`, `Mul`, `Index`, `Neg`, `BitAnd`, ...).

8.7 Generics or trait objects?

- Default to **generics** (`impl Trait` / `<T: Trait>`): they're zero-cost, and the compiler knows the concrete type.
- Use **`dyn Trait`** for heterogeneous collections, for plugin points chosen at runtime, to cut compile times or binary size, or when a generic parameter would leak into many unrelated types.

- Use an **enum** when the set of types is closed and known up front. It is often better than both.

> **WATCH OUT**
>
> Don't model Go-style "accept interfaces" by putting every dependency behind `Box<dyn Trait>` by reflex. Abstraction in Rust has costs you can see in the code: lifetime and `Send` bounds, dyn-compatibility rules, generic parameters that spread. Abstract where you need a seam, for example for testing or for swappable backends, and use concrete types elsewhere.

CHAPTER 9

Strings and Collections

9.1 `String` and `&str`

Go has one string type: an immutable, cheaply copyable byte sequence that is UTF-8 by convention. Rust has two main ones, which follow the owned/borrowed split from Chapter 4:

Type	Owns data?	Mutable?	Analogy
`String`	yes (heap)	yes	`strings.Builder` you can also read
`&str`	no	no	Go `string` (a borrowed view)

```rust
let lit: &str = "hello";            // &'static str, in the binary
let mut s: String = String::from("hello");
s.push_str(", world");
s.push('!');
let view: &str = &s;                // borrow the whole String
let part: &str = &s[0..5];          // byte-range slice: "hello"
let owned: String = part.to_string(); // or .to_owned(), String::from
```

Rust strings are **guaranteed valid UTF-8.** That's a type invariant, not a convention. Raw bytes are `Vec<u8>` / `&[u8]`, and converting from bytes is checked: `String::from_utf8(bytes)` returns a `Result`. (Go will happily put invalid UTF-8 into a `string`.)

Paths and OS strings have their own types: `PathBuf` / `&Path` and `OsString` / `&OsStr`. Unix file names don't have to be valid UTF-8, so they don't fit in `String`.

Indexing

```rust
let s = "naïve";
let c = s[2];        // error: strings cannot be indexed by integer
let b = s.as_bytes()[2];  // 0xC3, a raw byte
let sub = &s[0..2];       // "na": byte ranges are allowed...
let bad = &s[0..3];       // ...but this PANICS: not a char boundary
```

Go lets you index a string by byte (`s[2]`) and returns a byte. Rust refuses, because it's ambiguous whether you mean a byte or a character. You say which you mean:

```rust
for ch in s.chars() { }          // like Go's `for _, r := range s`
for (i, ch) in s.char_indices() { } // byte offset + char
for b in s.bytes() { }           // like `for i := 0; i < len(s); i++`
let n_chars = s.chars().count(); // O(n), like utf8.RuneCountInString
let len_bytes = s.len();         // O(1), bytes, like len(s)
```

Common operations

Go	Rust
`a + b`	`a + &b` (consumes `a`) or `format!("{a}{b}")`
`strings.Contains(s, x)`	`s.contains(x)`
`strings.HasPrefix(s, p)`	`s.starts_with(p)`
`strings.TrimPrefix(s, p)`	`s.strip_prefix(p)` → `Option<&str>`
`strings.Split(s, ",")`	`s.split(',')` → iterator
`strings.Fields(s)`	`s.split_whitespace()`
`strings.Cut(s, "=")`	`s.split_once('=')` → `Option<(&str, &str)>`
`strings.Join(xs, ", ")`	`xs.join(", ")`
`strings.ToUpper(s)`	`s.to_uppercase()`
`strings.TrimSpace(s)`	`s.trim()`
`strings.Replace(s, a, b, -1)`	`s.replace(a, b)`
`strconv.Itoa(n)`	`n.to_string()`
`strconv.Atoi(s)`	`s.parse::<i64>()`
`strings.Builder`	`String` + `push_str` / `write!`

Most of these return borrowed `&str` slices into the original instead of allocating, so they're cheap, and the borrow checker makes sure the original stays alive.

Which type to use

- Function parameters: `&str`.
- Struct fields and return values that create new text: `String`.
- Something that may or may not need to allocate: `Cow<'_, str>` (Chapter 11).
- To accept either: `impl Into<String>` (if you'll store it) or `impl AsRef<str>`.

9.2 `Vec<T>`

`Vec<T>` is Go's slice in its role as a growable array: a pointer, length and capacity, owning a heap buffer.

```rust
let mut v: Vec<i32> = Vec::new();
let mut w = Vec::with_capacity(100);  // make([]T, 0, 100)
let z = vec![0; 10];                  // make([]T, 10)
v.push(1);                            // append(v, 1)
v.extend([2, 3, 4]);                  // append(v, xs...)
v.extend_from_slice(&other);
let last = v.pop();                   // Option<i32>
v.insert(0, 9);  v.remove(0);         // O(n), like slices.Insert/Delete
v.truncate(2);   v.clear();
v.sort();  v.sort_by_key(|x| -x);  v.dedup();
v.contains(&3);  v.binary_search(&3);
v.iter().position(|&x| x == 3);       // slices.Index
let s: &[i32] = &v[1..];              // borrow a slice
```

Indexing out of bounds panics, just as in Go. `v.get(i)` returns an `Option<&T>` instead. Since a `Vec` owns its elements, removing one gives you ownership of it (`pop`, `remove`, `swap_remove`, `drain`).

The big difference from Go: **a `Vec` and the slices borrowed from it can't alias in surprising ways.** In Go, `b := a[:2]; b = append(b, x)` can overwrite `a[2]`. The Rust equivalent doesn't compile, because `b` would be a borrow of `a`, and `push` needs ownership or `&mut` of the `Vec` itself.

Other sequence types:

- `[T; N]`: a fixed-size array on the stack.
- `VecDeque<T>`: a ring buffer (queue, deque).
- `BinaryHeap<T>`: a priority queue (cf. `container/heap`, but generic).
- `Box<[T]>`: a fixed-length heap slice, smaller than a `Vec`.
- `smallvec` / `arrayvec` crates: small inline buffers that avoid heap allocation.

9.3 `HashMap<K, V>`

```rust
use std::collections::HashMap;

let mut ages: HashMap<String, u32> = HashMap::new();
ages.insert("ann".to_string(), 31);   // returns Option<old value>
if let Some(a) = ages.get("ann") { }  // m["ann"] with ok-check
let a = ages.get("bob").copied().unwrap_or(0); // zero-value default
ages.remove("ann");                   // delete(m, k), returns Option<V>
ages.contains_key("ann");
for (name, age) in &ages { }          // iteration order unspecified
```

There are a few differences from Go maps:

- **No zero value on missing keys.** `get` returns an `Option`, and `ages["bob"]` panics if the key is absent.
- **No nil map trap.** `HashMap::new()` doesn't allocate until the first insert, and there's no nil state that panics on write.
- **Keys must be `Eq + Hash`.** Derive both for your own key types.
- **Lookups take borrowed forms.** A `HashMap<String, _>` can be queried with a `&str`, thanks to the `Borrow` trait, so you don't need to allocate a `String` just to look something up.
- **The hasher resists HashDoS** (SipHash-1-3). It's slower than Go's AES-based hash for small keys. Crates such as `ahash`, `foldhash` or `rustc-hash` (`FxHashMap`) are faster when keys aren't attacker-controlled.

The entry API

This is the idiomatic way to do "insert or update":

```rust
// Word count. Go: counts[w]++
let mut counts: HashMap<&str, usize> = HashMap::new();
for w in text.split_whitespace() {
    *counts.entry(w).or_insert(0) += 1;
}

// Group by. Go: groups[k] = append(groups[k], v)
let mut groups: HashMap<char, Vec<&str>> = HashMap::new();
for w in words {
    let k = w.chars().next().unwrap();
    groups.entry(k).or_default().push(w);
}
```

`entry` hashes once and gives you a handle you can inspect (`Entry::Occupied` / `Entry::Vacant`) or fill in. Go needs two lookups for this, and sometimes a `nil` check on the inner slice.

Other maps and sets

- `BTreeMap<K, V>` / `BTreeSet<T>`: sorted by key and iterated in order, with range queries (`map.range(a..b)`). Go has nothing like this in the standard library.
- `HashSet<T>`: Go's `map[T]struct{}`, done properly, with `union`, `intersection` and `difference`.
- `indexmap` crate: a hash map that preserves insertion order.

9.4 Ownership and collections

Moving a value into a collection transfers ownership. Iterating comes in three flavours, according to what you want to do with the elements:

```rust
for x in &v { }       // x: &T      — read (v.iter())
for x in &mut v { }   // x: &mut T  — modify in place (v.iter_mut())
for x in v { }        // x: T       — consume v (v.into_iter())
```

Go's `for _, x := range v` copies each element into `x`, so writing to `x` has no effect, which is a classic bug. In Rust, `for x in &mut v { *x += 1 }` modifies the elements in place, and `for x in v` makes it explicit that `v` is consumed.

CHAPTER 10

Closures and Iterators

10.1 Closures

```go
add := func(a, b int) int { return a + b }
```
GO

```rust
let add = |a: i32, b: i32| a + b;
let add = |a, b| a + b;          // types usually inferred
let greet = |name: &str| {
    let msg = format!("hi {name}");
    println!("{msg}");
};
```
RUST

In Go, a closure captures variables **by reference**, and the GC keeps them alive as long as the closure is. A Rust closure has to fit into the ownership system, so each captured variable is captured in the least demanding way that works:

- by shared reference (`&x`), if the body only reads `x`;
- by mutable reference (`&mut x`), if it modifies `x`;
- by value (move), if it consumes `x`.

The `move` keyword makes a closure take **ownership** of everything it captures:

```rust
let name = String::from("ann");
let greet = move || println!("hi {name}"); // name moved into closure
// name is no longer usable here
std::thread::spawn(greet);                 // ok: closure is 'static
```
RUST

You need `move` whenever the closure outlives the current scope: threads, async tasks, closures returned from functions, callbacks you store. In prac-

tice that means almost everywhere Go would launch a goroutine with `go func() {…}()`.

> **COMING FROM GO**
>
> Go 1.22 fixed the loop-variable capture bug by giving each iteration a fresh variable. Rust never had that bug: a closure that borrowed the loop variable couldn't outlive the iteration, and a `move` closure gets its own copy.

Fn, FnMut, FnOnce

Every closure has its own unique, anonymous type, so you can't name a closure's type. Instead, closures implement one or more of three traits, depending on what they do with their captures:

Trait	Can be called	Body may…
`Fn`	many times, even concurrently	only read captures
`FnMut`	many times, needs `&mut`	mutate captures
`FnOnce`	once	consume captures

Every `Fn` is also `FnMut`, and every `FnMut` is also `FnOnce`. Functions that accept closures use the least restrictive bound they can:

```rust
fn apply_twice<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 { f(f(x)) }

fn repeat<F: FnMut()>(n: usize, mut f: F) {
    for _ in 0..n { f(); }
}

let mut count = 0;
repeat(3, || count += 1);    // FnMut: mutates `count`
println!("{count}");         // 3

fn run_later(f: impl FnOnce() + Send + 'static) { /* … */ }
```

Storing or returning closures follows the rules from Chapter 8:

```rust
fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n
}
```

```rust
// Heterogeneous storage needs a trait object:
let handlers: Vec<Box<dyn Fn(&Request) -> Response>> = vec![
    Box::new(index),          // plain functions work too
    Box::new(move |r| not_found(r, &path)),
];
```

Plain function pointers have a type you can name, `fn(i32) -> i32` (lowercase `fn`). Closures that capture nothing coerce to it.

10.2 Iterators

An iterator is anything that implements `Iterator`. It has one required method, `next(&mut self) -> Option<Item>`, plus dozens of provided **adapters**. Go 1.23's range-over-func iterators (`iter.Seq[T]`) are *push* iterators: the sequence calls your loop body. Rust iterators are *pull* iterators: the consumer calls `next()`. Pull iterators compose easily: you can zip two of them, stop early, or step through by hand.

```rust
let names: Vec<String> = users
    .iter()                              // Iterator<Item = &User>
    .filter(|u| u.active)
    .map(|u| u.name.to_uppercase())
    .take(10)
    .collect();
```

Three things to know:

Iterators are lazy. The adapters (`map`, `filter`, `take`, …) do nothing on their own. Work happens when a *consumer* pulls items: `collect`, `sum`, `count`, `for_each`, `fold`, `find`, `any`, `max_by_key`, or a `for` loop. Calling `map` and dropping the result does nothing, and the compiler warns about it.

Iterators are zero-cost. After monomorphisation and inlining, the chain above compiles to roughly the same loop you'd write by hand, often with bounds checks removed and sometimes vectorised. Go's `slices` and `maps` helpers plus range-over-func offer similar ergonomics, but the closures aren't always inlined.

The three `iter` methods follow ownership. `iter()` yields `&T`, `iter_mut()` yields `&mut T`, `into_iter()` yields `T` and consumes the collection.

The adapters you'll use most

```rust
xs.iter().map(f)                // transform
xs.iter().filter(|x| p(x))      // keep matching
xs.iter().filter_map(|x| x.ok())// map + drop Nones in one go
xs.iter().flat_map(|x| x.children.iter())
xs.iter().enumerate()           // (index, item)
xs.iter().zip(ys.iter())        // pairs
xs.iter().rev()                 // backwards
xs.iter().skip(2).step_by(3)
xs.iter().take_while(|x| **x < 10)
xs.iter().chain(ys.iter())
xs.windows(2)  xs.chunks(64)    // slice methods: overlapping / not
xs.iter().peekable()            // look ahead without consuming
xs.iter().copied()  .cloned()   // &T -> T
```

Consumers

```rust
let total: i64 = xs.iter().sum();
let n = xs.iter().filter(|x| x.ok).count();
let first_big = xs.iter().find(|x| x.size > 100);   // Option<&T>
let any_bad = xs.iter().any(|x| x.bad);
let oldest = users.iter().max_by_key(|u| u.age);    // Option<&User>
let csv = names.join(",");                          // on slices of str
let product = xs.iter().fold(1, |acc, x| acc * x);
```

`collect` is generic over its output. The target type, given by an annotation or turbofish, decides what gets built:

```rust
let v: Vec<_> = it.collect();
let s: HashSet<_> = it.collect();
let m: HashMap<_, _> = pairs.into_iter().collect();
let text: String = chars.into_iter().collect();
let all: Result<Vec<_>, _> = strs.iter().map(|s| s.parse::<i32>())
                                 .collect();
let v = it.collect::<Vec<_>>();     // "turbofish" syntax
```

The `Result` case is especially useful: it collects into `Ok(Vec)` if every element is `Ok`, and stops at the first `Err`.

Writing your own iterator

Implement `next` (Chapter 8 showed `Countdown`). For one-off cases, `std::iter::from_fn` builds an iterator from a closure, and `std::iter::successors` from a seed and a step function:

```rust
let mut state = (0u64, 1u64);
let fib = std::iter::from_fn(move || {
    let r = state.0;
    state = (state.1, state.0 + state.1);
    Some(r)
});
let first10: Vec<u64> = fib.take(10).collect();

let powers = std::iter::successors(Some(1u32), |x| x.checked_mul(2));
```

To let callers write `for x in &my_type`, implement `IntoIterator` for `&MyType`. Usually this just delegates to an inner `Vec`'s `iter()`.

> **NOTE**
>
> Should you use a loop or an iterator chain? Both are idiomatic. Chains suit transformations: map, filter, collect. Loops suit complex control flow or many side effects. A chain that needs `?` inside a closure, or early-returns from the outer function, is usually clearer as a loop.

CHAPTER 11

Smart Pointers and Interior Mutability

In Go, `*T` does everything: it shares, it mutates, it keeps a value alive, and it can point into the middle of a struct. Rust splits those roles across several types, each with its own costs and guarantees. This chapter covers that toolbox and when to use each piece.

You want…	Use
a single owner, data on the heap	`Box<T>`
shared ownership, one thread	`Rc<T>`
shared ownership, across threads	`Arc<T>`
to mutate through a shared reference, one thread	`Cell<T>`, `RefCell<T>`
to mutate shared data across threads	`Mutex<T>`, `RwLock<T>`, atomics
one-time lazy initialisation	`OnceCell`, `OnceLock`, `LazyLock`
a non-owning back-reference	`Weak<T>`
to borrow or own, decided at runtime	`Cow<'a, T>`

11.1 Box<T>

`Box<T>` puts a value on the heap and owns it. Dropping the box frees the allocation. It's a single pointer with no overhead. You use it in three situations:

```rust
// 1. Recursive types (otherwise the size would be infinite)
enum Expr {
    Num(f64),
    Add(Box<Expr>, Box<Expr>),
    Neg(Box<Expr>),
}
```

```
// 2. Trait objects
let handler: Box<dyn Fn(u32) -> u32> = Box::new(|x| x + 1);

// 3. Moving a large value around cheaply, or keeping a stable address
let big = Box::new([0u8; 1 << 20]);
```

Go does all of this with an ordinary pointer, and escape analysis decides whether the allocation goes on the heap. Rust makes heap allocation an explicit, visible decision.

11.2 `Rc<T>` and `Arc<T>`: shared ownership

When a value really has several owners, and no single one of them naturally outlives the others, use reference counting:

RUST

```
use std::rc::Rc;

let config = Rc::new(Config::load());
let a = Service::new(Rc::clone(&config));  // count = 2
let b = Service::new(Rc::clone(&config));  // count = 3
// Config is freed when the last Rc is dropped.
```

`Rc::clone` copies a pointer and increments a counter. It doesn't copy the `Config`. Writing `Rc::clone(&x)` rather than `x.clone()` makes it clear that the clone is cheap.

`Rc` isn't thread-safe because its counter isn't atomic, and the compiler won't let it cross threads (Chapter 13). `Arc<T>` ("atomically reference counted") is the thread-safe version, a little slower because of the atomic operations. Use `Arc` for anything shared between threads or async tasks.

The important restriction: **`Rc<T>` and `Arc<T>` give only shared (`&T`) access.** Shared XOR mutable still applies, so on its own an `Arc<T>` is read-only. To mutate, combine it with interior mutability.

> **COMING FROM GO**
>
> A Go pointer that several goroutines share is roughly an `Arc<Mutex<T>>`, except that Go doesn't make you use the `Mutex`. Rust makes you choose. A plain `Arc<T>` is immutable sharing and needs no lock. If you need mutation, the lock is part of the type, and you can't reach the data without taking it.

11.3 Interior mutability

Some designs really do need to mutate data that is shared: a cache inside a `&self` method, a counter shared by callbacks, a graph node updated by its neighbours. **Interior mutability** types allow mutation through a shared reference by moving the aliasing check from compile time to runtime, or by making the operation atomic.

`Cell<T>`

`Cell<T>` is for `Copy` types on a single thread. You `get` and `set` whole values and never borrow the inside, so nothing can alias:

```rust
use std::cell::Cell;

struct Stats { hits: Cell<u64> }

impl Stats {
    fn record(&self) {               // &self, yet mutates
        self.hits.set(self.hits.get() + 1);
    }
}
```

`RefCell<T>`

`RefCell<T>` applies the borrow rules **at runtime**. `borrow()` returns a shared guard and `borrow_mut()` an exclusive one. Breaking the rules, for example calling `borrow_mut()` while a `borrow()` guard is still alive, **panics**.

```rust
use std::cell::RefCell;
use std::rc::Rc;

let log = Rc::new(RefCell::new(Vec::<String>::new()));

let log2 = Rc::clone(&log);
let on_event = move |e: &str| log2.borrow_mut().push(e.to_string());

on_event("start");
on_event("stop");
println!("{:?}", log.borrow()); // ["start", "stop"]
```

`Rc<RefCell<T>>` is the single-threaded "shared mutable object" and the closest thing to a Go pointer to a struct. Its thread-safe counterpart is `Arc<Mutex<T>>` (or `Arc<RwLock<T>>`).

> **WATCH OUT**
>
> `RefCell` doesn't remove the borrow rules. It turns violations into panics. Keep guards short-lived (`cell.borrow_mut().push(x)` on one line), and don't hold one across a call that might borrow the same cell again. If your code is full of `Rc<RefCell<...>>`, consider whether a design with clearer ownership would be simpler.

Lazy and one-time initialisation

```rust
use std::sync::LazyLock;
use regex::Regex;

static EMAIL: LazyLock<Regex> =
    LazyLock::new(|| Regex::new(r"^[^@]+@[^@]+$").unwrap());

fn valid(s: &str) -> bool { EMAIL.is_match(s) }
```

`LazyLock` is Go's `sync.OnceValue`, and `OnceLock` is `sync.Once` plus the value it guards. `OnceCell` and `LazyCell` are the single-threaded versions.

11.4 Cycles and `Weak<T>`

Reference counting can't free cycles. If two `Rc`s point at each other, neither count ever reaches zero, and both values **leak**. Leaks are memory *safe* in Rust, but you still don't want them. Go's tracing GC handles cycles; Rust's reference counting doesn't.

`Weak<T>` is a non-owning pointer that doesn't keep the value alive. You call `upgrade()`, which returns an `Option<Rc<T>>`, to get access. The usual shape is a tree where parents own their children and children hold a weak pointer back to the parent:

```rust
use std::rc::{Rc, Weak};
use std::cell::RefCell;

struct Node {
```

```
    value: i32,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}
```

11.5 Graphs: use indices

Graph-shaped data with arbitrary edges is where Go's GC is most convenient, and where `Rc<RefCell<Node>>` code is at its worst. The idiomatic Rust approach is to **store nodes in a `Vec` and refer to them by index**:

```rust
struct Graph {
    nodes: Vec<Node>,
}

struct Node {
    name: String,
    edges: Vec<usize>,     // indices into Graph::nodes
}

impl Graph {
    fn add(&mut self, name: &str) -> usize {
        self.nodes.push(Node { name: name.into(), edges: vec![] });
        self.nodes.len() - 1
    }
    fn link(&mut self, a: usize, b: usize) {
        self.nodes[a].edges.push(b);
    }
}
```

The borrow checker is happy because the `Graph` owns every node and edges are plain integers. It's also cache-friendly. The risk is stale indices after a removal, which crates like `slotmap` (generational indices) and `petgraph` handle. This **arena pattern** is everywhere in Rust: compilers, ECS game engines, DOM implementations.

11.6 `Cow`: borrow or own

`Cow<'a, B>` ("clone on write") holds either a borrowed reference or an owned value. It's useful for functions that usually return their input unchanged but sometimes need to allocate:

```rust
use std::borrow::Cow;

fn normalize(s: &str) -> Cow<'_, str> {
    if s.contains('\t') {
        Cow::Owned(s.replace('\t', "    "))  // allocate only if needed
    } else {
        Cow::Borrowed(s)
    }
}
```

`Cow<str>` dereferences to `&str`, so callers mostly don't care which one they got.

11.7 `Deref` and deref coercion

Smart pointers implement `Deref`, so `*boxed` gives you the inner value, method calls pass straight through, and the compiler inserts conversions automatically where types require it:

```rust
fn takes_str(s: &str) {}

let s = String::from("x");
let b = Box::new(String::from("y"));
let r = Rc::new(String::from("z"));
takes_str(&s);  // &String     -> &str
takes_str(&b);  // &Box<String> -> &String -> &str
takes_str(&r);  // &Rc<String>  -> &String -> &str
```

This chain of conversions is **deref coercion**. It's why `&String` passes for `&str` and `&Vec<T>` for `&[T]`.

> **WATCH OUT**
>
> Don't use `Deref` to imitate Go struct embedding (`impl Deref for Wrapper { Target = Inner }`) so that a wrapper "inherits" the inner type's methods. It works, but it's considered an anti-pattern: it confuses readers, and trait implementations don't come along with it. Forward the methods you need explicitly, or expose the inner value.

CHAPTER 12

Modules, Crates and Testing

12.1 Modules are not directories

In Go, a package is a directory and every file in it is part of the package. In Rust, a **crate** is a tree of **modules** rooted at `lib.rs` or `main.rs`. Modules are declared explicitly in code, and the file layout follows those declarations.

OUTPUT

```
src/
├── lib.rs
├── config.rs
└── net/
    ├── mod.rs        (or: src/net.rs, alongside the net/ directory)
    ├── client.rs
    └── server.rs
```

RUST

```rust
// src/lib.rs
pub mod config;   // loads src/config.rs
pub mod net;      // loads src/net.rs or src/net/mod.rs

// src/net/mod.rs (or src/net.rs)
pub mod client;   // loads src/net/client.rs
mod server;       // private module
```

A file that isn't reached by a `mod` declaration **isn't compiled at all**. This trips up nearly everyone at first. A module can also be written inline: `mod tests { … }`.

So a Go package maps loosely onto a Rust module, and a Go module (one `go.mod`) onto a Cargo package. Modules are much lighter than Go packages, though: they're namespaces inside one compilation unit, they can refer to each other cyclically, and they're often small, a single file or even an inline block.

12.2 Visibility

Items are private to their module (and its children) by default:

Go	Rust
`lowercase` (package-private)	no modifier (module-private)
`Uppercase` (exported)	`pub`
`internal/` packages	`pub(crate)`
—	`pub(super)`, `pub(in path)`

`pub(crate)` is the workhorse: visible anywhere in this crate, invisible to users of the crate. Struct fields have their own visibility, so `pub struct User { pub name: String, hash: String }` exposes the type but keeps `hash` private.

12.3 Paths and use

```rust
use std::collections::HashMap;
use std::io::{self, Read, Write};
use crate::config::Config;        // absolute path from crate root
use super::helpers;               // parent module
use self::client::Client;         // this module (rarely needed)
use serde::{Deserialize, Serialize};

pub use crate::net::client::Client; // re-export at a nicer path
```

`use` only creates a local alias. It doesn't load anything. External crates listed in `Cargo.toml` are available by name everywhere. A common convention: import types directly (`HashMap`), and bring functions in through their parent module (`io::stdin()`, `fs::read()`), so that the origin stays visible at the call site.

Re-exporting with `pub use` lets a crate expose a flat public API (`mylib::Client`) while keeping a deep internal structure. Go can't do this, because the import path *is* the directory layout.

12.4 Cargo.toml

```toml
[package]
name = "inventory"
version = "0.3.1"
edition = "2024"
rust-version = "1.85"      # minimum supported Rust (MSRV)

[dependencies]
serde = { version = "1", features = ["derive"] }
tokio = { version = "1.40", features = ["full"] }
anyhow = "1"
internal-lib = { path = "../internal-lib" }
fork = { git = "https://github.com/me/fork", branch = "fix" }

[dev-dependencies]          # tests, examples, benches only
proptest = "1"

[features]
default = ["json"]
json = ["dep:serde_json"]

[dependencies.serde_json]
version = "1"
optional = true
```

Some differences from Go modules:

- **Version requirements are ranges.** `"1.40"` means `>=1.40.0, <2.0.0`. Cargo picks the *newest* compatible version, unlike Go's minimal version selection, and records the choice in `Cargo.lock`. Commit `Cargo.lock` for applications. For libraries it's optional; current guidance leans towards committing it there too.
- **Different major versions can coexist** in one build (`rand 0.7` and `rand 0.8`). Their types are distinct and won't unify, which occasionally causes confusing errors. Go handles this through `/v2` import paths.
- **Crates come from crates.io**, a central registry, not from VCS URLs. A published version is immutable.
- **Features** are compile-time switches that enable optional code and dependencies, somewhat like build tags, but declared and additive. Code checks them with `#[cfg(feature = "json")]`.

Useful commands: `cargo add`, `cargo update`, `cargo tree` (like `go mod graph`), `cargo tree -d` (show duplicate versions), `cargo doc --open` (docs for your *dependencies* too), and `cargo install` (like `go install`).

12.5 Workspaces

A workspace is a set of packages sharing one `Cargo.lock` and one `target/` directory, much like `go.work`, and commonly used for monorepos:

```toml
# Cargo.toml at the repository root
[workspace]
members = ["crates/*"]
resolver = "3"

[workspace.dependencies]
serde = { version = "1", features = ["derive"] }
```

Member crates then write `serde = { workspace = true }`. Splitting a large project into several crates in a workspace is also the main way to **speed up compilation**, because the crate is the unit of parallel and incremental compilation.

12.6 Conditional compilation

```rust
#[cfg(target_os = "linux")]
fn platform() -> &'static str { "linux" }

#[cfg(not(target_os = "linux"))]
fn platform() -> &'static str { "other" }

if cfg!(debug_assertions) { /* debug builds only */ }
```

This replaces Go's `//go:build` lines and `_linux.go` file suffixes. The attribute can go on any item, including a single function, a `use` or a whole `mod`.

12.7 Testing

Testing works much as in Go: it's built in, unit tests sit next to the code, and `cargo test` runs everything.

```rust
pub fn add(a: i32, b: i32) -> i32 { a + b }

#[cfg(test)]                // compiled only for `cargo test`
mod tests {
    use super::*;           // see the parent module's private items

    #[test]
    fn adds() {
        assert_eq!(add(2, 2), 4);
    }

    #[test]
    #[should_panic(expected = "overflow")]
    fn overflows() { add(i32::MAX, 1); }

    #[test]
    fn parses() -> Result<(), std::num::ParseIntError> {
        let n: i32 = "42".parse()?;   // tests may return Result
        assert!(n > 0, "expected positive, got {n}");
        Ok(())
    }
}
```

Go	Rust
`foo_test.go`, same package	`#[cfg(test)] mod tests` in the same file
`package foo_test` (black-box)	`tests/*.rs` integration tests
`t.Errorf` / `t.Fatalf`	`assert!`, `assert_eq!` (they panic)
`t.Run` subtests, table tests	loop over cases, or `rstest` crate
`go test -run Name`	`cargo test name` (substring filter)
`t.Parallel()`	the default; `--test-threads=1` to disable
`Example*` functions	doc tests in `///` comments
`testing.B` benchmarks	`criterion` or `divan` crates
`go test -fuzz`	`cargo fuzz`
`testdata/`	any path; `env!("CARGO_MANIFEST_DIR")`

Tests run **in parallel threads by default**, so tests that share global state (environment variables, the current directory, a database) need their own coordination, for example the `serial_test` crate.

There's no `t.Errorf` that records a failure and keeps going. Assertions panic, so the first failure stops the test. For table tests, put the case name in the assertion message:

```rust
#[test]
fn table() {
    let cases = [("1", Some(1)), ("x", None), ("-2", Some(-2))];
    for (input, want) in cases {
        let got = input.parse::<i32>().ok();
        assert_eq!(got, want, "input {input:?}");
    }
}
```

Doc tests are code blocks in documentation comments. They're compiled and run by `cargo test`, which keeps examples in the docs working:

```rust
/// Adds two numbers.
///
/// ```
/// assert_eq!(mylib::add(1, 2), 3);
/// ```
pub fn add(a: i32, b: i32) -> i32 { a + b }
```

Mocking is less common than in Go. With no structural interfaces, you can't define a narrow interface in the test package after the fact. The usual approaches are a trait at the seam with a fake implementation in tests, generic parameters, or the `mockall` crate. Many Rust codebases favour pure functions and integration tests over heavy mocking.

CHAPTER 13

Concurrency with Threads

Go makes concurrency cheap and easy to start: write `go f()`, share memory or pass messages, and run `-race` in CI to catch what you got wrong. Rust makes it **checked**. The ownership rules you've already learned extend to threads, and a program with a data race doesn't compile.

13.1 Threads are OS threads

```rust
use std::thread;

let handle = thread::spawn(|| {
    // runs on a new OS thread
    expensive_computation()
});
let result = handle.join().unwrap(); // Err if the thread panicked
```

`thread::spawn` creates a real OS thread, with a megabyte-scale stack and a kernel-level context switch. It isn't a goroutine. Thousands of threads are fine; a million aren't. For Go-style "one task per connection" at scale, Rust uses `async` with a runtime such as Tokio (next chapter). For CPU-bound parallelism it uses a thread pool such as Rayon (below).

`spawn` requires a `'static` closure (Chapter 5), so the thread can't borrow from the spawning function. Move data in:

```rust
let data = vec![1, 2, 3];
let h = thread::spawn(move || data.iter().sum::<i32>());
// `data` moved into the thread; unusable here
println!("{}", h.join().unwrap());
```

Unlike a goroutine, a thread can **return a value** through its `JoinHandle`, so there's no need for a result channel or a variable captured from outside.

13.2 `Send` and `Sync`

Two marker traits carry all of Rust's thread safety:

- **`Send`**: ownership of a value of this type can be transferred to another thread.
- **`Sync`**: a `&T` can be shared between threads. Formally, `T` is `Sync` if `&T` is `Send`.

You almost never implement them. The compiler derives them **automatically** from a type's fields: a struct made of `Send` fields is `Send`. Nearly everything is both. These are the notable exceptions:

Type	Send	Sync	Why
`Rc<T>`	no	no	non-atomic reference count
`Cell<T>`, `RefCell<T>`	yes	no	unsynchronised interior mutation
`MutexGuard<T>`	no	yes	must unlock on the locking thread
raw pointers	no	no	compiler can't reason about them
`Arc<T>`	yes	yes	only if `T: Send + Sync`
`Mutex<T>`	yes	yes	only if `T: Send`; the lock adds `Sync`

(The "yes" entries for wrapper types assume `T` itself is `Send` / `Sync`.)

`thread::spawn` requires its closure to be `Send`. Try to move an `Rc` into a thread and you get:

OUTPUT

```
error[E0277]: `Rc<Vec<i32>>` cannot be sent between threads safely
```

KEY IDEA

Data race freedom in Rust is not a separate feature. It follows directly from *aliasing XOR mutability* plus `Send` / `Sync`. A data race needs two threads accessing the same memory, at least one of them writing, without synchronisation. Ownership rules out unsynchronised shared mutable access everywhere, and threads are just one more place where the rule applies.

13.3 Shared state: `Arc<Mutex<T>>`

The classic racy Go program:

```go
count := 0
var wg sync.WaitGroup
for i := 0; i < 10; i++ {
    wg.Add(1)
    go func() { defer wg.Done(); count++ }() // DATA RACE
}
wg.Wait()
```

The direct translation doesn't compile, since `count` can't be borrowed mutably by ten threads at once. The correct version:

```rust
use std::sync::{Arc, Mutex};
use std::thread;

let count = Arc::new(Mutex::new(0));
let mut handles = vec![];
for _ in 0..10 {
    let count = Arc::clone(&count);
    handles.push(thread::spawn(move || {
        *count.lock().unwrap() += 1;
    }));
}
for h in handles { h.join().unwrap(); }
println!("{}", *count.lock().unwrap()); // 10
```

`Arc` provides shared ownership across threads, and `Mutex` provides synchronised mutation. Note where the data lives.

The mutex owns the data

```go
type Cache struct {
    mu    sync.Mutex
    items map[string]string // protected by mu (by convention!)
}
```

```rust
struct Cache {
    items: Mutex<HashMap<String, String>>,
}

impl Cache {
    fn get(&self, k: &str) -> Option<String> {
```

```
        let items = self.items.lock().unwrap(); // MutexGuard
        items.get(k).cloned()
    } // guard dropped => unlocked
}
```

In Go, a comment tells you that `mu` protects `items`. In Rust, `Mutex<T>` **contains** the `T`, and the only way to reach it is `lock()`, which returns a guard. The guard dereferences to `&mut T`, and dropping it unlocks the mutex. You can't touch the data without the lock, and you can't forget to unlock. Note that `get` takes `&self`. `Mutex` is an interior-mutability type (Chapter 11), so many threads can share `&Cache`.

Poisoning: if a thread panics while holding the lock, the mutex is marked *poisoned* and later `lock()` calls return `Err`, because the data may be half-updated. Most code just calls `.unwrap()`. The `parking_lot` crate provides faster mutexes without poisoning.

Other primitives: `RwLock<T>` (`sync.RWMutex`), `Condvar` (`sync.Cond`), `Barrier`, and `std::sync::atomic::{AtomicU64, AtomicBool, …}` (`sync/atomic`). For the counter above, an `AtomicU64` with `fetch_add(1, Ordering::Relaxed)` is the better choice. Atomics take an explicit memory **ordering**. Go's atomics are always sequentially consistent (`SeqCst`). When unsure, use `SeqCst`, and use `Relaxed` for plain counters.

> **WATCH OUT**
>
> Rust prevents data races, not **deadlocks** or higher-level race conditions. Lock ordering still matters. Watch out for temporaries: in `match m.lock().unwrap().get(k) { … }` the guard stays alive until the end of the whole `match`, which can lead to self-deadlock if you lock again inside an arm. Bind the result to a variable first.

13.4 Scoped threads: borrowing across threads

`thread::scope` lets threads **borrow** local data, because the scope guarantees that every thread joins before it returns. It's the Rust version of the `sync.WaitGroup` pattern, with the `Wait()` built in:

```rust
let mut chunks = vec![vec![1, 2], vec![3, 4], vec![5, 6]];
let total = 7;

thread::scope(|s| {
    for chunk in &mut chunks {
        s.spawn(|| {                    // no `move`, no Arc
            for x in chunk.iter_mut() { *x *= total; }
        });
    }
}); // all threads joined here

println!("{chunks:?}"); // [[7, 14], [21, 28], [35, 42]]
```

Each thread gets an exclusive `&mut` to its own chunk, and they all share `&total`. The borrow checker confirms that the chunks don't overlap. Go can't check this.

13.5 Channels

```rust
use std::sync::mpsc;   // "multi-producer, single-consumer"

let (tx, rx) = mpsc::channel();         // unbounded
// let (tx, rx) = mpsc::sync_channel(10); // bounded, like make(chan T, 10)

for id in 0..3 {
    let tx = tx.clone();
    thread::spawn(move || {
        tx.send(format!("worker {id} done")).unwrap();
    });
}
drop(tx); // drop our copy so the channel can close

for msg in rx {           // like `for msg := range ch`
    println!("{msg}");
}
```

There are two differences from Go channels.

Closing is ownership. There's no `close(ch)`. A channel closes when every `Sender` has been dropped. Then `recv()` returns `Err` and the receiving loop ends. This removes Go's "send on closed channel" panic and the question

of who is supposed to close the channel. The `drop(tx)` above is the Rust counterpart of a `close` after `wg.Wait()`. Forgetting it is the Rust counterpart of a goroutine blocked forever on `range ch`.

Sending moves the value. After `tx.send(v)`, `v` belongs to the receiver, so the sender can't keep mutating something it has just handed off. That's "share memory by communicating" enforced by the type system.

The standard library has no `select`. For thread-based code, the `crossbeam-channel` crate provides MPMC channels and a `select!` macro. In async code, Tokio's channels and `tokio::select!` fill this role (next chapter).

13.6 Data parallelism with Rayon

For CPU-bound work, the Rayon crate turns sequential iterators into parallel ones with a work-stealing pool:

```rust
use rayon::prelude::*;

let sum: u64 = (0..10_000_000u64)
    .into_par_iter()
    .filter(|n| n % 3 == 0)
    .map(|n| n * n)
    .sum();

let mut images = load_all();
images.par_iter_mut().for_each(|img| img.resize(800, 600));
```

The `Send` / `Sync` bounds on the closures ensure that this is safe. If the closure captured an `Rc` or mutated shared state without a lock, it wouldn't compile. In Go you'd build a worker pool with goroutines, a channel and a `WaitGroup`.

13.7 Global state

```rust
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::{LazyLock, Mutex};

static REQUESTS: AtomicUsize = AtomicUsize::new(0);
static REGISTRY: LazyLock<Mutex<Vec<String>>> =
    LazyLock::new(|| Mutex::new(Vec::new()));
```

```
fn handle() {
    REQUESTS.fetch_add(1, Ordering::Relaxed);
    REGISTRY.lock().unwrap().push("x".into());
}
```

A `static` must be `Sync`, because any thread can reach it. `static mut` exists but needs `unsafe` to touch, and the 2024 edition denies references to it by default. Use atomics or locks instead. A package-level `var` in Go, by contrast, is shared mutable state with no protection at all.

13.8 Summary

Go	Rust
`go f()`	`thread::spawn` (OS thread) / `tokio::spawn` (task)
`sync.WaitGroup`	`JoinHandle::join`, `thread::scope`
`sync.Mutex` + comment	`Mutex<T>` that owns `T`
`sync.RWMutex`	`RwLock<T>`
`sync/atomic`	`std::sync::atomic` (explicit ordering)
`sync.Once`, `OnceValue`	`OnceLock`, `LazyLock`
`chan T` (buffered)	`mpsc::sync_channel`, crossbeam, tokio::sync::mpsc
`close(ch)`	drop all senders
`select`	`crossbeam::select!`, `tokio::select!`
`go test -race`	the compiler
worker pool	`rayon`

CHAPTER 14

Async Rust

This is where Go and Rust differ most, and the chapter where your Go intuition is most likely to mislead you.

14.1 Two designs

Go has *stackful* coroutines. Each goroutine has its own growable stack, and the runtime scheduler multiplexes them onto OS threads. Blocking operations park the goroutine and the runtime handles it. Every function can block, so there's no distinction between sync and async code. The cost is a mandatory runtime, a few KB of stack per goroutine, and no control over scheduling.

Rust has *stackless* coroutines. An `async fn` is compiled into a **state machine**: a struct holding the local variables that are live at each `.await`, plus a `poll` method that advances it. There's no built-in runtime. You pick an **executor** library, nearly always **Tokio**, which drives these state machines on a thread pool with an epoll/kqueue/io_uring reactor. A task costs exactly the size of its state machine, often a few hundred bytes, and a single allocation.

What you give up is uniformity. Async functions can only be awaited from other async code, a blocking call inside async code stalls the executor thread, and there are extra concepts to learn: `Future`, `Send` bounds, `Pin`. This is the "function colouring" problem. Go doesn't have it; Rust does.

14.2 Hello, Tokio

```toml
[dependencies]
tokio = { version = "1", features = ["full"] }
```

```rust
use std::time::Duration;
use tokio::time::sleep;

async fn fetch(id: u32) -> String {
    sleep(Duration::from_millis(100)).await;
    format!("item {id}")
}

#[tokio::main]
async fn main() {
    let a = fetch(1).await;        // sequential
    println!("{a}");

    let handle = tokio::spawn(fetch(2)); // like `go`, returns a handle
    let b = handle.await.unwrap();       // Err if the task panicked
    println!("{b}");
}
```

`#[tokio::main]` is a macro that sets up the runtime and runs `main`'s future to completion on it. `tokio::spawn` is the counterpart of `go`: it schedules an independent **task** that runs concurrently and possibly in parallel. Tokio's default runtime is multi-threaded and work-stealing, like Go's scheduler.

14.3 Futures are lazy

> **KEY IDEA**
>
> Calling an `async fn` **does nothing** except build a future. Work happens only when the future is `.await`ed, or handed to an executor with `spawn`. This is the opposite of `go f()`, which starts immediately.

```rust
let fut = fetch(3);   // nothing has happened yet
// ... later ...
let s = fut.await;    // now it runs
```

The `Future` trait is small:

```rust
pub trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
        -> Poll<Self::Output>; // Poll::Ready(v) or Poll::Pending
}
```

The executor calls `poll`. If the future can't make progress, for instance because a socket isn't readable yet, it registers the `Waker` from `cx` with the reactor and returns `Pending`. When the socket becomes readable, the reactor wakes the task and the executor polls it again. `.await` is syntax for "poll this inner future, and if it's pending, return pending from me too". You'll almost never implement `poll` yourself, but this model explains everything else in this chapter.

14.4 Concurrency without spawning

Because futures are values, you can run several **concurrently within one task**, with no spawning and no `'static` requirement:

```rust
// Run both, wait for both (like errgroup without goroutines)
let (user, orders) = tokio::join!(get_user(id), get_orders(id));

// Same, but stop at the first error
let (user, orders) = tokio::try_join!(get_user(id), get_orders(id))?;

// Race: first one wins, the others are DROPPED (cancelled)
tokio::select! {
    res = fetch_primary() => handle(res),
    res = fetch_replica() => handle(res),
    _ = sleep(Duration::from_secs(2)) => log("timeout"),
}
```

Futures inside `join!` can borrow local variables, because they don't outlive the enclosing function. `tokio::spawn` requires `'static` (move owned data in, or use `Arc`), for the same reason `thread::spawn` does.

For a dynamic number of tasks, use `JoinSet`, which replaces the `WaitGroup` + results channel pattern:

```rust
use tokio::task::JoinSet;

let mut set = JoinSet::new();
for url in urls {
    set.spawn(async move { download(&url).await });
}
while let Some(res) = set.join_next().await {
    let body = res??; // JoinError, then the task's own error
    process(body);
}
// Dropping a JoinSet aborts any tasks still running.
```

14.5 Cancellation

In Go, cancellation is cooperative and explicit. You thread a `context.Context` through every call and check `ctx.Done()`. In Rust async, **dropping a future cancels it**. The next time it would have been polled, it simply isn't. Its locals are dropped, running their destructors, so sockets close and guards release.

```rust
use tokio::time::{timeout, Duration};

match timeout(Duration::from_secs(5), fetch_report()).await {
    Ok(report) => save(report),
    Err(_) => eprintln!("timed out"), // fetch_report() was dropped
}
```

No `ctx` parameter is needed. The losing branches of a `select!`, an aborted `JoinHandle`, and a dropped `JoinSet` are all cancelled the same way.

> **WATCH OUT**
>
> Cancellation can happen at **any** `.await`. If a function does `let x = read_header().await; let y = read_body().await;` and is cancelled between the two awaits, the header is consumed and lost. This is called **cancellation safety**, and it matters most in `select!` loops, where a branch's future is dropped every time another branch wins. Tokio's docs mark which methods are cancel-safe. For cancellation that needs a graceful shutdown, use `tokio_util::sync::CancellationToken`, which is the closest thing to `context.WithCancel`.

A spawned task is **not** cancelled when its `JoinHandle` is dropped. It keeps running detached, like a goroutine. Call `handle.abort()` to cancel it.

14.6 The `Send` requirement

On the multi-threaded runtime, a task may move between threads at any `.await`, so `tokio::spawn` requires the future to be `Send`. A future is `Send` only if everything it holds **across an await point** is `Send`. This produces one of the most common async errors:

```rust
async fn bad(state: Arc<std::sync::Mutex<State>>) {
    let guard = state.lock().unwrap();
    do_io().await;              // guard still alive here
    guard.update();
}
// error: future cannot be sent between threads safely
//   ... `MutexGuard<'_, State>` is not `Send`
//   ... held across an await
```

There are two fixes. Usually the best one is not to hold the guard across the await:

```rust
async fn good(state: Arc<std::sync::Mutex<State>>) {
    let snapshot = {
        let g = state.lock().unwrap();
        g.snapshot()
    };                          // guard dropped here
    let result = do_io_with(snapshot).await;
    state.lock().unwrap().apply(result);
}
```

Otherwise, use `tokio::sync::Mutex`, whose `lock()` is itself async and whose guard is `Send`. It's slower, so use it only when you really must hold a lock across an await. The same applies to `Rc` and `RefCell` across awaits: use `Arc` and a mutex, or restructure.

14.7 Don't block the executor

The Go runtime notices when a goroutine is stuck in a blocking syscall and moves other goroutines to another thread. Tokio can't do that. If a task

calls `std::thread::sleep`, does synchronous file I/O, takes a contended `std::sync::Mutex` for a long time, or runs a 200 ms computation, it holds up every other task scheduled on that worker thread.

- Use the async versions: `tokio::time::sleep`, `tokio::fs`, `tokio::net`, async database drivers.
- Move blocking or CPU-heavy work off the executor:

```rust
let hash = tokio::task::spawn_blocking(move || {
    bcrypt::hash(password, 12)   // CPU-heavy, synchronous
}).await??;
```

A rule of thumb: code shouldn't run longer than about 10–100 µs between `.await` points.

14.8 Async channels and sync primitives

`tokio::sync` has the async counterparts of the earlier primitives:

Go	`tokio::sync`
`make(chan T, n)`	`mpsc::channel(n)` (bounded)
one-shot result channel	`oneshot::channel()`
fan-out to all listeners	`broadcast::channel(n)`
"latest value" + notify	`watch::channel(v)`
`sync.Mutex`	`Mutex` (async `lock().await`)
semaphore via buffered chan	`Semaphore`
`sync.Cond`	`Notify`

```rust
use tokio::sync::{mpsc, oneshot};

enum Cmd {
    Get { key: String, reply: oneshot::Sender<Option<String>> },
    Set { key: String, val: String },
}

// The "actor" pattern: one task owns the state, others send it messages.
async fn store(mut rx: mpsc::Receiver<Cmd>) {
    let mut map = std::collections::HashMap::new();
```

```
    while let Some(cmd) = rx.recv().await {
        match cmd {
            Cmd::Get { key, reply } => {
                let _ = reply.send(map.get(&key).cloned());
            }
            Cmd::Set { key, val } => { map.insert(key, val); }
        }
    }
}
```

The actor pattern maps directly onto a Go goroutine that owns its state and serves requests from a channel. In Rust it has the added advantage that the state needs no lock at all.

14.9 A TCP echo server, side by side

GO

```
ln, _ := net.Listen("tcp", ":7000")
for {
    conn, err := ln.Accept()
    if err != nil { continue }
    go func() {
        defer conn.Close()
        io.Copy(conn, conn)
    }()
}
```

RUST

```
use tokio::net::TcpListener;

#[tokio::main]
async fn main() -> std::io::Result<()> {
    let ln = TcpListener::bind("0.0.0.0:7000").await?;
    loop {
        let (mut sock, _) = ln.accept().await?;
        tokio::spawn(async move {
            let (mut r, mut w) = sock.split();
            let _ = tokio::io::copy(&mut r, &mut w).await;
        }); // sock dropped (closed) when the task ends
    }
}
```

The two programs have the same shape. The differences are in the details: `.await` marks each suspension point, `async move` moves ownership of the socket into the task, and `Drop` takes the place of `defer conn.Close()`.

14.10 Other things you'll meet

- **Async functions in traits** work natively (since Rust 1.75). If the resulting futures must be `Send` for use with `dyn Trait` or across spawns, the `async-trait` or `trait-variant` crates help.
- **Async closures**, `async || { ... }`, are stable since 1.85.
- `Pin` appears because a state machine may contain references into itself, so it must not move in memory once polled. `Box::pin(fut)` and the `pin!` macro handle this. You'll see `Pin<Box<dyn Future<Output = T>>>`, the type-erased future, in trait objects and recursive async functions.
- **Streams** (`futures::Stream`, `tokio_stream`) are async iterators. Use `while let Some(x) = stream.next().await`.

14.11 Threads or async?

- **Async** for I/O-bound work with many concurrent operations: servers, clients, proxies, anything that would have thousands of goroutines in Go.
- **Threads** (plus Rayon) for CPU-bound work, or for a handful of long-running workers. Sync code is simpler: no colouring, no `Send` puzzles.
- Mixing them is normal: an async server that offloads hashing to `spawn_blocking` or to a Rayon pool.

> **COMING FROM GO**
>
> The biggest mental shift is that **there is no runtime unless you bring one**, and its behaviour, such as blocking, task placement or cancellation, is library code you can read and configure. Once you accept that futures are lazy values and that dropping a future cancels it, most of the remaining surprises in async Rust follow from those two facts.

CHAPTER 15

A Tour of the Ecosystem

Go's standard library includes an HTTP server, JSON, templating, SQL interfaces and crypto. Rust's standard library is deliberately small: collections, I/O, threads, networking primitives, and not much else. Everything else comes from crates. That sounds worrying, but a stable core set of crates has become the de facto standard library. This chapter tours the ones you'll use in nearly every service.

15.1 serde: serialisation

Go uses struct tags and reflection. Rust uses **derive macros** that generate serialisation code at compile time, so there's no reflection and no runtime cost.

GO

```go
type User struct {
    ID      int64    `json:"id"`
    Name    string   `json:"name"`
    Email   *string  `json:"email,omitempty"`
    Roles   []string `json:"roles"`
}
```

RUST

```rust
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
struct User {
    id: i64,
    name: String,
    #[serde(skip_serializing_if = "Option::is_none")]
    email: Option<String>,
    #[serde(default)]
    roles: Vec<String>,
```

```
}

let u: User = serde_json::from_str(r#"{"id":1,"name":"ann"}"#)?;
let s = serde_json::to_string_pretty(&u)?;
```

`serde` is format-agnostic. The same derives work with `serde_json`, `toml`, `serde_yaml`, `bincode`, `rmp-serde` (MessagePack), `csv` and others.

Some differences that matter in practice:

- **Missing fields are errors** unless the field is an `Option` or marked `#[serde(default)]`. Go silently fills in zero values.
- **Unknown fields are ignored** by default. Add `#[serde(deny_unknown_fields)]` to reject them, the counterpart of `DisallowUnknownFields`.
- **Enums map to tagged JSON.** `#[serde(tag = "type")]` turns `enum Event { Click { x: i32 }, Key { code: u32 } }` into `{"type":"Click","x":5}`, which takes custom `UnmarshalJSON` code in Go.
- For dynamic JSON, use `serde_json::Value`, the counterpart of `map[string]any`.

15.2 clap: command-line parsing

RUST

```rust
use clap::Parser;

/// Resize images in bulk.
#[derive(Parser)]
#[command(version)]
struct Args {
    /// Input directory
    input: std::path::PathBuf,
    /// Target width in pixels
    #[arg(short, long, default_value_t = 800)]
    width: u32,
    /// Print each file as it is processed
    #[arg(short, long)]
    verbose: bool,
}
```

```
fn main() {
    let args = Args::parse(); // --help generated from doc comments
}
```

`clap` covers what `flag` plus `cobra` do in Go: subcommands (as an enum), environment-variable fallbacks, validation and shell completions.

15.3 tracing: logging and spans

`tracing` is the counterpart of `log`/`slog`, with structured fields and *spans*: contexts that attach to everything logged inside them, including across `.await` points.

RUST

```
use tracing::{info, instrument, warn};

#[instrument(skip(db))]                 // span with the args as fields
async fn handle(db: &Db, user_id: u64) -> Result<()> {
    info!(attempt = 1, "loading user");
    if db.is_locked().await {
        warn!("db locked");
    }
    Ok(())
}

fn main() {
    tracing_subscriber::fmt().json().init(); // like slog.NewJSONHandler
}
```

The simpler `log` crate, with `env_logger`, still exists and is fine for small tools.

15.4 HTTP: axum and reqwest

axum (built on Tokio and `hyper`) is the most widely used web framework. Handlers are plain async functions, and their arguments are *extractors* that parse parts of the request:

RUST

```
use axum::{Json, Router, extract::{Path, State}, routing::get};
use std::sync::Arc;

#[derive(Clone)]
```

```
struct AppState { db: Arc<Db> }

async fn get_user(
    State(st): State<AppState>,
    Path(id): Path<u64>,
) -> Result<Json<User>, AppError> {
    let user = st.db.find_user(id).await?;
    Ok(Json(user))
}

#[tokio::main]
async fn main() {
    let state = AppState { db: Arc::new(Db::connect().await) };
    let app = Router::new()
        .route("/users/{id}", get(get_user))
        .with_state(state);
    let ln = tokio::net::TcpListener::bind("0.0.0.0:3000").await.unwrap();
    axum::serve(ln, app).await.unwrap();
}
```

Compared with `net/http`, there's more type machinery, but the `Path` parsing, JSON decoding and error-to-response conversion are type-checked. `AppError` implements axum's `IntoResponse` trait to turn errors into HTTP status codes. Middleware comes from `tower` (timeouts, rate limits, tracing, CORS), and these layers can be reused across many frameworks.

reqwest is the HTTP client:

RUST

```
let user: User = reqwest::Client::new()
    .get(format!("{base}/users/{id}"))
    .bearer_auth(token)
    .timeout(Duration::from_secs(5))
    .send().await?
    .error_for_status()?      // non-2xx -> Err (Go makes you check)
    .json().await?;
```

15.5 Databases: sqlx

`sqlx` is async, and its `query!` macros **check SQL against a real database schema at compile time**:

```rust
let rows = sqlx::query_as!(
    User,
    "SELECT id, name, email FROM users WHERE org_id = $1",
    org_id
)
.fetch_all(&pool)
.await?;
```

If a column is renamed, this fails to compile. If `email` is nullable, the macro requires `Option<String>` in `User`. (For offline builds and CI, run `cargo sqlx prepare` to cache the query metadata.) There are alternatives: `diesel` (a type-safe query builder), `sea-orm` (an ORM), and `tokio-postgres` / `rusqlite` (direct drivers).

15.6 Choosing crates

There's no single standard, but these signals are reliable: download counts on crates.io, documentation on docs.rs, the `blessed.rs` and `lib.rs` indexes, and whether the crate is maintained by a well-known group (tokio-rs, rust-lang, RustCrypto, rustls). Keep an eye on your dependency tree. `cargo tree` and `cargo deny` play the roles of `go mod graph` and `govulncheck`, and `cargo audit` checks the RustSec advisory database.

CHAPTER 16

Macros

Go avoids metaprogramming, and you use `go generate` when you really need it. Rust has macros as a core feature. You've been using them since page one: `println!`, `vec!`, `format!`, `#[derive(Debug)]`. You don't need to write many yourself, but you should know what kinds exist and what they can do.

16.1 Declarative macros: `macro_rules!`

These are pattern-matching rewrite rules over syntax:

```rust
macro_rules! hashmap {
    ( $( $k:expr => $v:expr ),* $(,)? ) => {{
        let mut m = ::std::collections::HashMap::new();
        $( m.insert($k, $v); )*
        m
    }};
}

let ages = hashmap! { "ann" => 31, "bob" => 27 };
```

`$k:expr` matches an expression, and `$( … ),*` matches a comma-separated repetition. Other fragment types include `ident`, `ty`, `pat`, `block`, `tt` (any single token tree) and `literal`. Macros are *hygienic*: a variable defined inside a macro can't clash with the caller's variables.

Macros run before type checking, so they can do things functions can't:

- take a variable number of arguments (`vec![1, 2, 3]`);
- check format strings at compile time (`println!`);
- generate items, such as many similar `impl` blocks for different types;

- capture source information (`file!()`, `line!()`, `stringify!(expr)`, `include_str!("schema.sql")`).

Reach for a macro only once a function or generic can't do the job. Macro errors are harder to read and IDE support is weaker.

16.2 Procedural macros

Procedural macros are Rust functions that run **inside the compiler**. They take a token stream and return new code. There are three kinds:

```rust
#[derive(Serialize)]          // derive: adds impls for a type
struct S { /* ... */ }

#[tokio::main]                // attribute: rewrites the item
async fn main() {}

let q = sqlx::query!("SELECT 1"); // function-like: arbitrary DSL
```

They live in a separate crate with `proc-macro = true`, and they're usually written with the `syn` (parsing) and `quote` (code generation) crates. This is how `serde`, `clap`, `thiserror`, `tokio::main`, `sqlx::query!` and `tracing::instrument` work.

> **COMING FROM GO**
>
> A procedural macro does the job of `go generate` + `stringer` + `mockgen` + struct-tag reflection, but it runs as part of the normal build. There are no generated files to commit and nothing goes stale. The price is compile time, since proc-macro crates (and `syn`) must be built first.

To see what a macro expands to, use `cargo expand` (install it with `cargo install cargo-expand`). The rust-analyzer "Expand macro" action does the same in your editor.

16.3 Attributes you'll see often

```rust
#[derive(Debug, Clone)]       // derive impls
#[cfg(test)]                  // conditional compilation
#[test]                       // mark a test
```

```
#[allow(dead_code)]             // silence a lint locally
#[must_use]                     // warn if the return value is ignored
#[inline]                       // inlining hint (cross-crate)
#[non_exhaustive]               // callers must add a `_` match arm
#[deprecated(note = "use foo")] // like // Deprecated: comments
#[repr(C)]                      // C-compatible layout (FFI)
```

`#[non_exhaustive]` on a public enum or struct lets a library add variants or fields later without a breaking change. Downstream `match`es must then include a wildcard arm.

CHAPTER 17

Unsafe Rust and FFI

17.1 What `unsafe` means

Everything so far is **safe Rust**, where the compiler guarantees memory safety and freedom from data races. Some things can't be checked by the compiler: calling C, implementing a data structure with raw pointers, or talking to hardware. For these there's `unsafe`, which unlocks five extra abilities:

1. dereferencing raw pointers (`*const T`, `*mut T`);
2. calling `unsafe` functions, including all foreign (C) functions;
3. accessing or modifying `static mut` variables;
4. implementing `unsafe` traits (such as `Send` / `Sync` by hand);
5. accessing fields of `union`s.

That's all. `unsafe` doesn't switch off the borrow checker or type checking. It marks a block where **you** guarantee invariants that the compiler can't check, and if you get them wrong the result is undefined behaviour.

```rust
let x = 42;
let p = &x as *const i32;   // creating raw pointers is safe
let y = unsafe { *p };      // dereferencing is not
```

Go's equivalent is the `unsafe` package (`unsafe.Pointer`, `unsafe.Slice`), used for similar reasons. The difference is in culture and tooling. In Rust, `unsafe` blocks are kept small and commented with a `// SAFETY:` note explaining why the invariants hold. They're wrapped in safe APIs, and they're checked with **Miri**, an interpreter that detects undefined behaviour (`cargo +nightly miri test`).

```rust
/// Returns the element at `i` without bounds checking.
///
/// # Safety
/// `i` must be less than `slice.len()`.
pub unsafe fn get_fast<T>(slice: &[T], i: usize) -> &T {
    // SAFETY: the caller guarantees `i < slice.len()`.
    unsafe { slice.get_unchecked(i) }
}
```

The standard library is safe abstractions built over small `unsafe` cores: `Vec`, `String`, `Mutex`, `Arc`. Most application code never needs `unsafe`. If you find yourself writing it to get around the borrow checker, step back and redesign, because that's almost always the wrong fix.

17.2 Calling C

```rust
use std::ffi::{CString, c_char, c_int};

unsafe extern "C" {
    fn strlen(s: *const c_char) -> usize;
    safe fn abs(x: c_int) -> c_int;  // declared safe to call (2024)
}

fn main() {
    let s = CString::new("hello").unwrap(); // NUL-terminated copy
    let n = unsafe { strlen(s.as_ptr()) };
    println!("{n} {}", abs(-3));
}
```

Compared with cgo:

- **No call overhead.** Rust and C share a calling convention and a stack, and there's no goroutine stack switch. An FFI call costs about as much as a normal function call. cgo calls cost tens of nanoseconds and complicate scheduling.
- **Bindings are generated** by `bindgen` from C headers. `cbindgen` goes the other way, generating headers for Rust code.
- **Exporting to C** is easy, so Rust can replace C inside an existing codebase one library at a time:

```rust
#[unsafe(no_mangle)]
pub extern "C" fn add(a: i32, b: i32) -> i32 { a + b }
```

Build it with `crate-type = ["cdylib"]` or `["staticlib"]` under `[lib]` in `Cargo.toml`. This is why Rust is common in Python extensions (PyO3, maturin), Node addons (napi-rs), WebAssembly (`wasm-bindgen`), and inside C/C++ projects such as the Linux kernel, Android and browsers. Go's runtime makes those uses awkward.

17.3 Layout control

```rust
#[repr(C)]          // C-compatible field order and padding
struct Header { magic: u32, len: u16, flags: u16 }

#[repr(u8)]         // enum discriminant is a u8
enum Kind { A = 1, B = 2 }

#[repr(transparent)] // same layout as the single field
struct Fd(i32);
```

Without `#[repr(C)]`, the compiler is free to reorder struct fields to reduce padding. Go keeps fields in declaration order, so in Go you reorder them by hand. To see a type's size, use `std::mem::size_of::<T>()`.

CHAPTER 18

Translating Go Idioms

This chapter takes Go patterns you use every day and shows how each one is usually expressed in Rust. Translating line by line produces poor Rust, so the goal here is to translate the *intent*.

18.1 Interfaces for dependency injection

```go
type Store interface {
    Get(ctx context.Context, id string) (*Item, error)
}
type Service struct{ store Store }
```

In Rust, the choice is between generics and trait objects:

```rust
trait Store {
    async fn get(&self, id: &str) -> Result<Item>;
}

// Generic: zero-cost, concrete type known at compile time
struct Service<S: Store> { store: S }

// Trait object: one Service type, store chosen at runtime
struct DynService { store: Arc<dyn DynStore> }
```

Prefer the generic version when the store is fixed per binary and only swapped in tests. Choose `dyn` when you'd otherwise have generics spreading through dozens of types. Note that a trait with `async fn` isn't directly dyn-compatible today, hence `DynStore`, whose methods return boxed futures. The `async-trait` crate generates that for you.

18.2 `context.Context`

`Context` does three jobs in Go, and Rust splits them up:

- **Cancellation**: dropping the future (Chapter 14), `select!` with a `CancellationToken`, or `tokio::time::timeout`.
- **Deadlines**: `timeout(dur, fut)` at the call site, rather than a deadline carried through every call.
- **Request-scoped values**: explicit parameters or a request struct. For logging and trace IDs, `tracing` spans propagate the context automatically.

There's no need to add a `ctx` parameter to every function.

18.3 `defer`

Use `Drop` (Chapter 3). For one-off cleanup that isn't tied to a type, the `scopeguard` crate provides `defer!`:

```rust
let _guard = scopeguard::guard((), |_| println!("cleanup"));
```

Name the binding `_guard`, not `_`. A bare `_` drops the value immediately, so the cleanup runs right away. This is a classic bug, and it applies to lock guards too.

18.4 Functional options → builders

```go
srv := NewServer(addr, WithTimeout(5*time.Second), WithTLS(cfg))
```

```rust
let srv = Server::builder(addr)
    .timeout(Duration::from_secs(5))
    .tls(cfg)
    .build()?;
```

```rust
pub struct ServerBuilder { addr: String, timeout: Duration,
                           tls: Option<TlsConfig> }

impl ServerBuilder {
    pub fn timeout(mut self, d: Duration) -> Self {
        self.timeout = d;
        self
```

```
    }
    pub fn tls(mut self, c: TlsConfig) -> Self {
        self.tls = Some(c);
        self
    }
    pub fn build(self) -> Result<Server> { /* validate, construct */ }
}
```

Each method takes `self` by value and returns it, so calls chain. For simple cases, a `Config` struct that implements `Default` and is filled in with `..Default::default()` is shorter. The `bon` and `typed-builder` crates can derive builders.

18.5 Embedding → composition

Rust has no struct embedding and no inheritance. Use plain fields and write the delegating methods yourself. Share behaviour through traits with default methods:

RUST

```
trait Named {
    fn name(&self) -> &str;
    fn greeting(&self) -> String {        // default method
        format!("Hello, {}", self.name())
    }
}
```

When a Go struct embeds another to "inherit" an interface implementation, in Rust you implement the trait on the outer type and forward to the inner field. It's a few more lines, and there's no ambiguity about which method gets called.

18.6 `any` and type switches

- A closed set of types → an `enum`.
- "Any type with behaviour X" → generics or `dyn Trait`.
- Truly dynamic data (JSON of unknown shape) → `serde_json::Value`.
- A heterogeneous registry keyed by type → `HashMap<TypeId, Box<dyn Any>>`. This is rare.

18.7 `iota`, `String()`, `stringer`

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
enum Level { Debug, Info, Warn, Error }

impl std::fmt::Display for Level {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        let s = match self {
            Level::Debug => "debug", Level::Info => "info",
            Level::Warn => "warn",   Level::Error => "error",
        };
        f.write_str(s)
    }
}
```

The `strum` crate derives `Display`, `FromStr` and iteration over all variants. `Level::Warn as u8` gives you the discriminant.

18.8 `init()` and package state

Rust has no `init()` and no code runs before `main`. Initialise state explicitly in `main` and pass it down, or use `LazyLock` for lazily built globals (Chapter 13). It's more explicit, and the initialisation order can't surprise you.

18.9 Worker pools

Go	Rust
N goroutines reading from a jobs channel	`rayon` `par_iter` (CPU)
goroutine per item + `WaitGroup`	`JoinSet` (async)
buffered channel as semaphore	`tokio::sync::Semaphore`
`errgroup.Group` with `SetLimit`	`futures::stream::iter(..).buffer_unordered(n)`

```rust
use futures::{StreamExt, stream};

let results: Vec<_> = stream::iter(urls)
    .map(|u| async move { fetch(&u).await })
```

```
    .buffer_unordered(16)        // at most 16 in flight
    .collect()
    .await;
```

18.10 Go gotchas that don't exist in Rust

- **Typed nil interfaces** (`err != nil` when `err` holds a nil `*MyErr`). There's no nil.
- **Copying a `sync.Mutex` by value.** Moves and the missing `Copy` impl make this impossible.
- **Ranging over a value copy** and writing to it. `iter_mut` exists for that.
- **Map writes during iteration**, or concurrent map writes. These don't compile.
- **Forgotten `err` checks.** `Result` is `#[must_use]`, and the value can't be reached without handling the error.
- **Shadowed `err` in an inner scope.** No `err` variables at all.
- **Slices sharing a backing array after `append`.** The borrow checker rules this out.
- **Goroutine leaks on abandoned channels.** Tasks still leak if you're careless, but dropping a future cancels it, and channels close when their senders are dropped.

18.11 Mistakes Go developers make in Rust

1. **Using `&String`, `&Vec<T>` or `&Box<T>` parameters.** Take `&str`, `&[T]` or `&T`.
2. **Putting references in long-lived structs.** Own the data instead.
3. **Writing `Arc<Mutex<...>>` around everything.** Start with clear ownership and pass `&mut`. Share only what really needs to be shared.
4. **Modelling OOP hierarchies with traits.** Reach for enums first.
5. **Calling `unwrap()` everywhere in library code.** Propagate with `?` and use `expect("why this can't fail")` for invariants.
6. **Blocking inside async code.** Use `spawn_blocking`.
7. **Fighting the borrow checker with `unsafe`.** Don't.
8. **Taking `clone()` as defeat.** Cloning a small `String` at a boundary is fine. Profile before you optimise.
9. **Ignoring Clippy.** Run `cargo clippy` often. It teaches idioms.

18.12 Where to go next

- *The Rust Programming Language* (“the book”), free at doc.rust-lang.org. Use it as a reference for anything this book skims.
- *Rust for Rustaceans* by Jon Gjengset, for intermediate topics: API design, unsafe, async internals.
- *Rust Atomics and Locks* by Mara Bos, the definitive text on Rust concurrency, free online.
- The Tokio tutorial at tokio.rs, before you write your first service.
- The Rustonomicon, before you write your first `unsafe` block.
- *Rustlings* exercises and Advent of Code, to drill the borrow checker until it becomes second nature.

The quickest way to learn is to rewrite a small Go tool you know well, such as a CLI, a little HTTP service, or a log parser, in Rust, and to read every compiler error carefully. The messages are good, and they often tell you exactly what to change.

Appendix A: Phrasebook

Quick translations of common Go constructs. "≈" marks an approximation.

Declarations

Go	Rust
`x := 1`	`let x = 1;`
`var x int` (zero value)	`let x: i32 = 0;` (explicit)
`x = 2` (reassign)	`let mut x = 1; x = 2;`
`const N = 10`	`const N: usize = 10;`
`type ID int64`	`struct Id(i64);`
`type Alias = T`	`type Alias = T;`
`var global = ...`	`static GLOBAL: T = ...;`
`func f(a int) (int, error)`	`fn f(a: i32) -> Result<i32, E>`
`func (s *S) M()`	`impl S { fn m(&mut self) {} }`
`func NewS() *S`	`fn new() -> Self`
`go:embed file.txt`	`include_str!("file.txt")`

Types

Go	Rust
`string`	`String` (owned), `&str` (borrowed)
`[]byte`	`Vec<u8>`, `&[u8]`
`[]T`	`Vec<T>` (owned), `&[T]` / `&mut [T]`
`[N]T`	`[T; N]`
`map[K]V`	`HashMap<K, V>`, `BTreeMap<K, V>`
`map[T]struct{}`	`HashSet<T>`

Go	Rust
`*T` (maybe nil)	`Option<Box<T>>`, `Option<&T>`
`*T` (shared, mutated)	`&mut T`, `Rc<RefCell<T>>`, `Arc<Mutex<T>>`
`interface{ M() }`	`trait Tr { fn m(&self); }`
interface value	`Box<dyn Tr>`, `&dyn Tr`
`any`	generics, an `enum`, or `Box<dyn Any>`
`error`	`Box<dyn Error>`, `anyhow::Error`, an enum
`func(int) int`	`fn(i32) -> i32`, `impl Fn(i32) -> i32`
`chan T`	`mpsc::Sender<T>` + `Receiver<T>`
`struct{}`	`()` or a unit struct
`[T any]`	`<T>`
`[T comparable]`	`<T: Eq + Hash>`
`[T cmp.Ordered]`	`<T: Ord>`

Control flow

Go	Rust
`for i := 0; i < n; i++`	`for i in 0..n`
`for i, v := range s`	`for (i, v) in s.iter().enumerate()`
`for k, v := range m`	`for (k, v) in &m`
`for {}`	`loop {}`
`for cond {}`	`while cond {}`
`switch x { case 1, 2: }`	`match x { 1 | 2 => .., _ => .. }`
`switch v := x.(type)`	`match` on an enum
`if v, ok := m[k]; ok`	`if let Some(v) = m.get(&k)`
`if err != nil { return err }`	`?`
`cond ? a : b` (n/a in Go)	`if cond { a } else { b }`
`defer f()`	`Drop` / scope guard
`panic(x)` / `recover()`	`panic!(..)` / `catch_unwind` (rare)
`goto`	labelled `break` / `continue`

Common operations

Go	Rust
`len(s)`	`s.len()`
`append(s, x)`	`v.push(x)`
`append(a, b...)`	`a.extend_from_slice(&b)`
`copy(dst, src)`	`dst.copy_from_slice(src)`
`s[a:b]`	`&s[a..b]`
`make([]T, n)`	`vec![T::default(); n]`
`make([]T, 0, n)`	`Vec::with_capacity(n)`
`m[k] = v`	`m.insert(k, v);`
`delete(m, k)`	`m.remove(&k);`
`sort.Slice(s, less)`	`s.sort_by(|a, b| a.x.cmp(&b.x))`
`slices.Sort(s)`	`s.sort()` / `s.sort_unstable()`
`slices.Contains(s, x)`	`s.contains(&x)`
`slices.Reverse(s)`	`s.reverse()`
`slices.Max(s)`	`s.iter().max()`
`maps.Keys(m)`	`m.keys()`
`new(T)`	`Box::new(T::default())`
`&T{...}`	`T { .. }` (or `Box::new(T { .. })`)
`x.(T)`	`match` / `downcast_ref::<T>()`
`errors.New("x")`	`anyhow!("x")` or an enum variant
`fmt.Errorf("ctx: %w", err)`	`.context("ctx")?`
`errors.Is(err, ErrX)`	`matches!(e, E::X)`
`os.Exit(1)`	`std::process::exit(1)`
`os.Args`	`std::env::args()`
`os.Getenv("K")`	`std::env::var("K")` → `Result`
`time.Now()`	`Instant::now()` / `SystemTime::now()`
`time.Since(t)`	`t.elapsed()`
`time.Sleep(d)`	`thread::sleep(d)` / `tokio::time::sleep(d).await`

Concurrency

Go	Rust
`go f()`	`thread::spawn(f)` / `tokio::spawn(fut)`
`ch := make(chan T, n)`	`let (tx, rx) = mpsc::sync_channel(n);`
`ch <- v`	`tx.send(v)`
`v := <-ch`	`rx.recv()`
`v, ok := <-ch`	`match rx.recv() { Ok(v) .., Err(_) .. }`
`close(ch)`	`drop(tx)`
`for v := range ch`	`for v in rx`
`select { ... }`	`tokio::select! { ... }`
`mu.Lock(); defer mu.Unlock()`	`let g = m.lock().unwrap();`
`wg.Add/Done/Wait`	`JoinHandle::join`, `thread::scope`
`ctx, cancel := WithTimeout(..)`	`tokio::time::timeout(d, fut)`
`atomic.AddInt64(&n, 1)`	`n.fetch_add(1, Ordering::Relaxed)`
`sync.Once`	`OnceLock` / `LazyLock`

Tooling

Go	Rust
`go mod tidy`	(automatic) / `cargo machete`
`go vet`	`cargo clippy`
`go test -race`	the compiler; Miri / loom for `unsafe`
`go test -bench`	`cargo bench` with criterion / divan
`go test -cover`	`cargo llvm-cov` / `cargo tarpaulin`
`go generate`	`build.rs` or proc macros
`pprof`	`perf`, `cargo flamegraph`, `samply`
`go build -ldflags "-s -w"`	`strip = true` in profile
`govulncheck`	`cargo audit` / `cargo deny`
`GOFLAGS=-tags=x`	`--features x`

Appendix B: Standard Library Map

Where to find the equivalent of each Go standard-library package. “std” means it’s in Rust’s standard library. Everything else is a crate on crates.io. Where several crates are listed, the first is the most common choice.

Go package	Rust
`fmt`	std: `format!`, `println!`, `std::fmt`
`strings`, `bytes`	std: methods on `str`, `[u8]`; `bstr`
`strconv`	std: `parse()`, `to_string()`
`unicode/utf8`	std: `char`, `str::chars`; `unicode-segmentation`
`errors`	std: `std::error::Error`; `thiserror`, `anyhow`
`sort`, `slices`, `maps`	std: slice and iterator methods; `itertools`
`container/heap`, `list`	std: `BinaryHeap`, `VecDeque`, `LinkedList`
`io`, `bufio`	std: `std::io` (`Read`, `Write`, `BufReader`)
`os`, `os/exec`	std: `std::fs`, `std::env`, `std::process`
`path/filepath`	std: `std::path` (`Path`, `PathBuf`)
`io/fs` walking	`walkdir`, `ignore`
`os/signal`	`tokio::signal`, `signal-hook`, `ctrlc`
`flag`	`clap`, `argh`, `lexopt`
`log`, `log/slog`	`tracing` + `tracing-subscriber`; `log`
`time`	std: `Duration`, `Instant`, `SystemTime`; `jiff`, `chrono`, `time`
`sync`, `sync/atomic`	std: `std::sync`; `parking_lot`, `crossbeam`
`context`	future dropping; `tokio_util::CancellationToken`
`net`	std: `std::net`; `tokio::net`
`net/http` (server)	`axum`, `actix-web`, `hyper`

Go package	Rust
`net/http` (client)	`reqwest`, `ureq` (blocking)
`net/http/httptest`	`axum-test`, `wiremock`, `tower::ServiceExt::oneshot`
`net/url`	`url`
`encoding/json`	`serde` + `serde_json`
`encoding/xml`	`quick-xml`
`encoding/csv`	`csv`
`encoding/base64`, `hex`	`base64`, `hex`
`encoding/binary`	std: `u32::from_le_bytes` etc.; `byteorder`, `zerocopy`
gob / protobuf / gRPC	`bincode`, `prost`, `tonic`
`text/template`, `html/template`	`askama`, `minijinja`, `tera`
`regexp`	`regex`
`crypto/*`	`rustls`, `ring`, `aws-lc-rs`, RustCrypto crates (`sha2`, `aes-gcm`, ...)
`crypto/rand`, `math/rand`	`rand`, `getrandom`
`math/big`	`num-bigint`, `rug`
`database/sql`	`sqlx`, `diesel`, `tokio-postgres`, `rusqlite`
`embed`	std: `include_str!`, `include_bytes!`; `rust-embed`
`testing`	std: `#[test]`; `rstest`, `proptest`, `insta`
`testing` benchmarks	`criterion`, `divan`
`runtime/pprof`	`pprof` crate, `perf`, `samply`
`reflect`	none; derive macros (`serde`, `strum`) instead
`unsafe`	std: `unsafe` blocks, `std::ptr`, `std::mem`
`plugin`	`libloading`, or WebAssembly (`wasmtime`)
`compress/gzip`, `zlib`	`flate2`
`archive/tar`, `zip`	`tar`, `zip`
`hash/crc32`, `fnv`	`crc32fast`, `fnv`, `xxhash-rust`
`html` parsing	`scraper`, `html5ever`
`image`	`image`

Go package	Rust
`os/user`	`uzers`, `whoami`
`syscall`, `x/sys`	`libc`, `nix`, `rustix`, `windows`